

Bachelorarbeit

zur Erlangung des Grades
Bachelor of Science (B. Sc.) in Informatik

Konzeption und prototypische Implementierung einer Rules-Engine für die automatisierte Erstellung von Asset Administration Shells

erstellt von **Luis Schweinberger**

Luis Schweinberger



luis.schweinberger@tha.de

Matrikelnummer:



**Technische Hochschule
Augsburg**

An der Hochschule 1
D-86161 Augsburg
T +49 821 5586-0
F +49 821 5586-3222
www.tha.de
info@tha.de

Erstprüfer	Prof. Dr.-Ing. Alexandra Teynor
Zweitprüfer	Prof. Dr. Phillip Heidegger
Themenvergabe am	25.05.2025
Eingereicht am	21.09.2025
Geheimhaltungsvereinbarung	Nein

Inhaltsverzeichnis

1	Einleitung	2
2	Grundlagen und Domänenwissen	4
2.1	Digitale Zwillinge	4
2.2	RAMI4.0	5
2.2.1	Objektwelten	5
2.2.2	Drei-Achsen-Modell	5
2.3	Asset Administration Shell	7
2.3.1	Typen der AAS	7
2.3.2	Typ- und Instanz-AAS	8
2.3.3	Submodels	8
2.3.4	SubmodelElements	8
2.3.5	Qualifiers	8
2.4	Eclipse Mnestix-Projekt	9
2.4.1	AAS-Browser	9
2.4.2	Eclipse BaSyx	9
2.4.3	Template Builder	10
2.4.4	AAS-Generator	10
3	Related Work	12
3.1	Literaturanalyse	12
3.2	Marktanalyse	14
3.2.1	AASX Server	15
3.2.2	Eclipse BaSyx	15
3.2.3	FA ³ ST Service	16
3.2.4	NOVAAS	16
3.3	Fazit	17
4	Anforderungsanalyse	18
4.1	Zweck und Scope	18
4.2	Begriffe und Referenzen	18
4.3	Systemkontext	19
4.4	Stakeholder	19
4.5	Betriebsumgebung, Annahmen und Abhängigkeiten	19
4.6	Schnittstellen und Datenquellen	20

4.7	Use-Cases	20
4.8	Qualitätsziele	21
4.9	Funktionale Anforderungen	22
4.10	Nicht-funktionale Anforderungen	23
4.11	Randbedingungen	23
4.12	Verifikationsmethoden	23
4.13	Akzeptanzkriterien (ACs)	24
4.14	Traceability	26
4.15	Zusammenfassung	26
5	Darstellung der Regeln	27
5.1	Ziel und Einordnung	27
5.2	Regelmodell	27
5.3	Notation der Submodels	28
5.4	Regeltypen mit Beispielen	28
5.4.1	Default-Werte (statisch)	28
5.4.2	Pfadregeln (dynamische Werte)	29
5.4.3	Listen-/Schleifenregeln (Duplizieren)	30
5.4.4	Filterregeln (bedingte Instanziierung)	31
5.4.5	Kardinalitätsregeln (Optional/Verpflichtend)	33
5.5	Zusammenfassung	34
6	Prototypische Implementierung	35
6.1	Pipeline-Architektur	35
6.1.1	Schritt 1: Aufteilung der Gesamtaufgabe	36
6.1.2	Schritt 2: Definition der Formate	36
6.1.3	Schritt 3: Implementierung der Kanäle	37
6.1.4	Schritt 4: Implementierung der Filter	38
6.1.5	Schritt 5: Fehlerbehandlung und Logging	39
6.1.6	Schritt 6: Zusammenstellen der Pipeline	39
6.1.7	Fazit	41
6.2	Integration in das Eclipse-Mnestix-Backend	41
6.2.1	Statische Integration	42
6.2.2	Dynamische Integration	43
6.3	Implementierung ausgewählter Pipeline-Schritte	45
6.3.1	DuplicateCollectionsAasGeneratorPipelineStep	47
6.3.2	MapDataToInstanceAasGeneratorPipelineStep	49
6.4	Implementierung der GUI	51
7	Evaluation	53
7.1	Verifikation durch Tests	53
7.1.1	Automatisierte Tests	53
7.1.2	Manuelle Tests	54

7.2	Verifikation durch Analyse und Demonstration	55
7.3	Diskussion der Ergebnisse	56
8	Fazit und Ausblick	58
8.1	Zusammenfassung	58
8.2	Erreichen der Zielsetzung	58
8.3	Mögliche Weiterentwicklungen	59
8.3.1	Vollständige AAS-Generierung	59
8.3.2	Erweiterte Regeltypen	59
8.3.3	Bearbeitung existierender Submodels	59
8.3.4	Nachvollziehbarkeit	60
8.3.5	Verbesserte Benutzeroberfläche	60
8.3.6	Erweiterte Datenverarbeitung	60
8.4	Alternative Ansätze	60
8.4.1	Semantisches Mapping	60
8.4.2	KI-gestützte Ansätze	61
8.5	Ausblick	61
	Anhang	62
	Abkürzungsverzeichnis	64
	Abbildungsverzeichnis	66
	Tabellenverzeichnis	67
	Codeverzeichnis	68
	Literaturverzeichnis	69

1 Einleitung

Auf der Hannover Messe 2011 wurde die *Industrie 4.0* als richtungsweisenden Trend für die zukünftige Entwicklung der deutschen Maschinenbau- und Automobilindustrie bewertet [1]. In den folgenden Jahren entstand aus der *Hightech-Strategie 2020* der Bundesregierung die *Plattform Industrie 4.0*, welche vom [Bundesministerium für Wirtschaft und Energie \(BMWE\)](#) (ehemalig [Bundesministerium für Wirtschaft und Klimaschutz \(BMWK\)](#)) und [Bundesministerium für Forschung, Technologie und Raumfahrt \(BMFTR\)](#) (ehemalig [Bundesministerium für Bildung und Forschung \(BMBF\)](#)) gemeinsam mit führenden Branchenvertreter:innen geleitet wird. Im Rahmen dieser Initiative wird durch die Entwicklung von Standards und dem Austausch von Know-How die digitale Transformation der deutschen Industrie vorangetrieben [2].

Industrie 4.0 beschreibt die vierte industrielle Revolution, welche durch die Digitalisierung und Vernetzung von Produktionsprozessen gekennzeichnet ist. Ziel ist es, intelligente Fabriken zu schaffen, in denen Maschinen, Anlagen und Produkte aktiv miteinander kommunizieren und sich autonom steuern können [3].

Ein zentrales Konzept der Industrie 4.0 ist der *Digitale Zwilling (Digital Twin)*, der die virtuelle Repräsentation eines physischen Objekts oder Systems darstellt [4]. Im *Leitbild Industrie 4.0* der Plattform Industrie 4.0 wird die vollständige Verbreitung standardisierter digitaler Zwillinge für die Umsetzung von Industrie 4.0 angestrebt [5].

Digitale Zwillinge ermöglichen die umfassende digitale Abbildung und Nachverfolgung von Produkten über ihren gesamten Lebenszyklus hinweg. Dies ist insbesondere für nachhaltige Produktionsweisen und die Umsetzung von Kreislaufwirtschaftskonzepten von zentraler Bedeutung, da so Informationen zu Materialzusammensetzung, Nutzung und Recyclingpotenzial effizient bereitgestellt werden können [6].

Mit der Einführung des [EU Digital Product Passport \(EU DPP\)](#) wird die digitale Dokumentation von Produkteigenschaften, Materialflüssen und Umweltauswirkungen für zahlreiche Produktkategorien ab 2027 beginnend mit dem Batteriepass phasenweise verpflichtend. [7], [8]. Digitale Zwillinge bieten hierfür eine geeignete technologische Grundlage, da sie die strukturierte Erfassung und den Austausch dieser Informationen unterstützen und so die Anforderungen des [EU DPP](#) effektiv umsetzbar machen.

Für einen branchenübergreifenden Informationsaustausch ist es essenziell, dass sich auf ein standardisiertes Kommunikationsformat geeignet wird. Die [Asset Administration Shell \(AAS\)](#) ist ein Standard für die Umsetzung von digitalen Zwillingen in der Industrie 4.0, der von der [Industrial Digital Twin Association \(IDTA\)](#) einem deutschen Interessenverband für digitale Zwillinge, entwickelt wird [9].

Derzeit werden viele [AASs](#) noch manuell erstellt, was sich im industriellen Maßstab als ineffizient und wenig skalierbar erweist. Dabei bestehen die zugrunde liegenden Daten häufig bereits in

homogen strukturierten Formaten, die sich für eine automatisierte Verarbeitung eignen.

Diese Arbeit untersucht, wie [AASS](#) automatisiert und regelbasiert aus solchen Datenquellen generiert werden können. Im Fokus steht die Entwicklung eines übertragbaren Regelwerks, das direkt innerhalb der [AAS](#) abgebildet werden kann. Dieses Regelwerk soll von einer *Rules-Engine* interpretiert und ausgeführt werden, deren Architektur im Rahmen dieser Arbeit konzipiert und prototypisch implementiert wird.

Zunächst werden die theoretischen Grundlagen zu [AASS](#), digitalen Zwillingen und relevanten Technologien in Kapitel 2 erläutert. Anschließend wird in Kapitel 3 der aktuelle Stand der Forschung und Open-Source-Software im Bereich der automatisierten Erstellung von [AASS](#) dargestellt. Darauf aufbauend werden in Kapitel 4 die Anforderungen an das Regelwerk und die Rules-Engine definiert. Kapitel 5 definiert die Funktionsweise und Semantik des Regelwerks. In Kapitel 6 wird ein minimaler Prototyp entwickelt, der in Kapitel 7 anhand der zuvor definierten Anforderungen evaluiert wird. Zuletzt wird in Kapitel 8 ein Fazit gezogen und ein Ausblick auf mögliche zukünftige Entwicklungen gegeben.

2 Grundlagen und Domänenwissen

Das Technologiefeld der Industrie 4.0 ist ein komplexes Gebiet, das verschiedene Technologien und Konzepte umfasst. Um die Entwicklung der Rules-Engine im Kontext der AAS zu verstehen, ist es wichtig, zunächst die grundlegenden Konzepte und Technologien zu erläutern, die in diesem Bereich relevant sind. Zunächst wird in Abschnitt 2.1 das grundlegende Konzept von Digitalen Zwillingen erklärt. Abschnitt 2.2 beschreibt anschließend das [Reference Architecture Model Industrie 4.0 \(RAMI4.0\)](#). Danach wird in Abschnitt 2.3 das darauf aufbauende Metamodell der AAS erläutert. Zuletzt wird in Abschnitt 2.4 ein Überblick über das *Eclipse-Mnestix*-Projekt gegeben, in dem die Rules-Engine implementiert werden soll.

2.1 Digitale Zwillinge

Im Rahmen der Industrie 4.0 wird häufig von *digitalen Zwillingen* gesprochen, sobald physische Objekte digital abgebildet werden. Bisher gibt es für den Begriff keine einheitliche Definition. In einer Literaturstudie von *Kritzinger et al.* [4] wurde eine Klassifizierung je nach Grad der Integration zwischen dem physischen bzw. geplanten Objekt und seinem digitalen Abbild als Definitionsgrundlage vorgeschlagen. Die Autor:innen sprechen von der folgenden Unterteilung:

- **Digital Model:** Ein Objekt wird digital abgebildet, ohne dass Daten automatisiert zwischen dem Objekt und dem Abbild übertragen werden.
- **Digital Shadow:** Das Objekt sendet automatisiert Informationen an das digitale Abbild, ohne dass Daten in die andere Richtung übertragen werden.
- **Digital Twin:** Zwischen dem Objekt und dem digitalen Abbild findet ein bidirektionaler Datenfluss statt, sodass Informationen des Objekts an das digitale Abbild gesendet werden und gleichzeitig das digitale Abbild das physische Objekt steuern kann.

Diese Definitionen wurden bereits in einigen nachfolgenden Werken aufgegriffen, jedoch noch nicht einheitlich standardisiert. Alle drei Konzepte haben aber das Ziel, Abläufe in der Industrie 4.0 vorherzusagen und zu optimieren [4], [10].

Die Funktionen, die im Rahmen dieser Arbeit entwickelt werden, erfüllen primär die Anforderungen an einen *Digital Shadow*.

2.2 RAMI4.0

Um das Konzept des digitalen Zwillings im Rahmen der Industrie 4.0 weiter zu präzisieren, kamen Expert:innen aus der deutschen Industrie und Forschung 2016 im Rahmen der *Plattform Industrie 4.0* zusammen und entwickelten das [Reference Architecture Model Industrie 4.0](#) [11]. Dieses Modell beschreibt eine Architektur, die verschiedene Aspekte der Industrie 4.0 integriert, darunter die digitale Repräsentation von Assets, deren Verwaltung und Interaktion.

Das Modell definiert zudem die sog. *I4.0-Komponente*, welche aus dem physischen Asset und seiner digitalen Repräsentation, der [AAS](#), besteht.

2.2.1 Objektwelten

Das [RAMI4.0](#) unterscheidet grundsätzlich zwischen der physischen Welt und der Informationswelt, wie in Abbildung 2.1 dargestellt.

Die physische Welt umfasst die realen Gegenstände, Produkte und Prozesse, aber auch lauffähige Software. Die Informationswelt bzw. virtuelle Welt hingegen bezieht sich auf die Repräsentation der Daten zu den physischen Objekten. Hier wird nochmals zwischen Modellwelt, Zustandswelt und Archivwelt unterschieden [11].

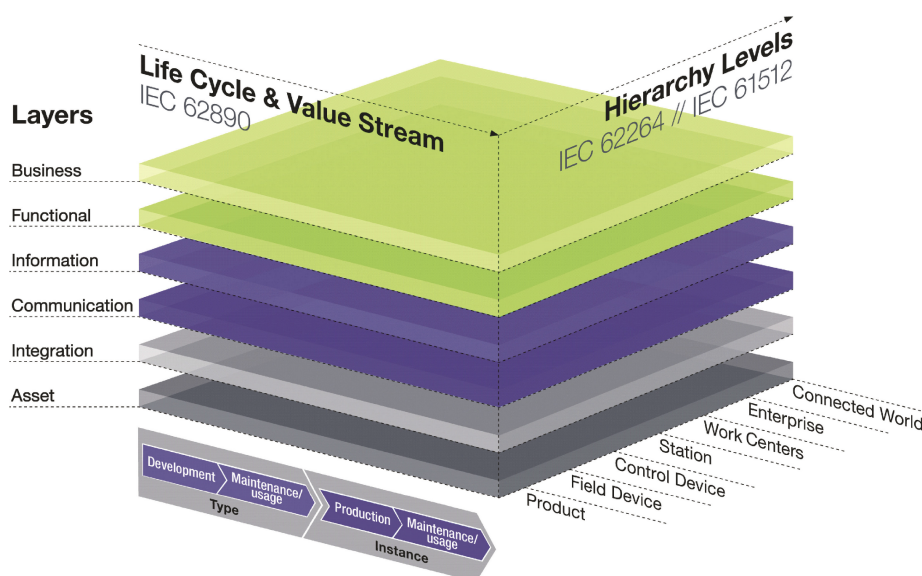
Im folgenden werden Objekte aus der physischen Welt als (*physische*) *Objekte* bezeichnet, während Objekte aus der Informationswelt als (*digitale*) *Repräsentationen* bezeichnet werden.

Modellwelt	Zustandswelt	Archivwelt	physische Welt
- Metadokumente (Standards, Normen, Richtlinien) - Technische Dokumentation (Funktionspläne, Anlagenpläne, Produktbeschreibungen, Prozedurbeschreibungen) - Operative Pläne (Produktionspläne, Projektpläne) - Geschäftsprozessbeschreibung	- Aktuelle Messwerte - Wirksame Sollwerte - Aktuelle Konfigurationsparameter	- Lebenszyklusdokumentation - Ereignisse - Zustandsverläufe - Projektverläufe - Prozesse	- Produkte - Hilfsmittel - Produktionsanlagen (Ausrüstungsgegenstände, Sensoren, Aktoren) - IT-Anlagen, Leitsysteme (Computer, Programme, Datenträger, Leitungen) - Schränke, Papier
Informationswelt			physische Welt

2.1 Abbildung: Objektwelten des RAMI4.0 [11]

2.2.2 Drei-Achsen-Modell

Das Drei-Achsen-Modell des [RAMI4.0](#) (siehe Abbildung 2.2) beschreibt die Merkmale eines Assets in strukturierter Form. Es besteht aus drei Dimensionen: der Architekturachse, der Hierarchieachse und der Lebenszyklusachse.



2.2 Abbildung: Drei-Achsen-Modell des RAMI4.0 [11]

Architekturachse

Zunächst werden Informationen innerhalb der Architekturachse in sechs *Layers* (Schichten) unterteilt. Nicht jede Schicht muss zwingend abgebildet werden. Der Austausch von Informationen kann nur über benachbarte Schichten erfolgen [11]:

- **Asset:** Diese Schicht repräsentiert das physische Asset und dessen Eigenschaften.
- **Integration:** Diese Schicht bindet das physische Asset in die digitale Welt ein, indem sie Schnittstellen und Protokolle bereitstellt.
- **Communication:** Kommunikationsprotokolle der I4.0-Komponente werden in dieser Schicht definiert.
- **Information:** In dieser Schicht werden Daten über das Asset gesammelt, die während der Benutzung des Assets anfallen, wie z.B. Betriebsdaten oder Zustandsinformationen.
- **Functional:** Hier wird die fachliche Funktionalität des Assets im Kontext seiner Rolle im Industrie 4.0-System beschrieben.
- **Business:** Diese Schicht umfasst geschäftliche Aspekte wie z.B. Auftragserteilungen oder monetäre Bedingungen.

Hierarchieachse

Des Weiteren können die Informationen in verschiedene *Hierarchiestufen* unterteilt werden. Diese orientieren sich an den Normen DIN EN 62264-1 und DIN EN 61512-1 zum Leitsystem eines Unternehmens und erweitern sie um die Bereiche, die für die Industrie 4.0 relevant sind. Die

Ebenen erstrecken sich von der vernetzten Welt, über eine Produktionslinie hin zu einer einzelnen Maschine [11].

Lebenszyklusachse

Zuletzt wird in der Lebenszyklusachse der Zustand des Assets in seinem Lebenszyklus beschrieben. Dabei wird in zwei Hauptphasen unterschieden: die *Typ-Phase* und die *Instanz-Phase*. Die Typ-Phase umfasst die Entwicklung, Planung und Konstruktion eines Assets, während die Instanz-Phase die Produktion, Nutzung, Wartung und Außerbetriebnahme des Assets abdeckt [11].

2.3 Asset Administration Shell

Wie in Abschnitt 2.2 erwähnt, wurde die AAS zuerst im RAMI4.0 als digitale Repräsentation eines physischen Assets definiert. Um eine einheitliche und standardisierte Beschreibung der AAS zu gewährleisten, wurde sie von der IDTA weiterentwickelt. Die IDTA ist ein Verein deutscher Unternehmen und Institute, der eine branchenweite offene Implementierung der AAS in der Industrie 4.0 anstrebt [12]. In mehreren Spezifikationen werden die Struktur, die Sicherheitsanforderungen oder das Extensible Markup Language (XML)-basierte Asset Administration Shell Datei-Format (AASX)-Format festgelegt [13]. Der Hauptfokus dieser Arbeit liegt auf dem „AAS-Metamodell“, welches den Aufbau der AAS definiert, ohne dabei bestimmte Technologien festzulegen. Es wurden inzwischen drei Major-Versionen der AAS veröffentlicht, die neueste Major-Version 3 wurde im November 2022 veröffentlicht und wird als Grundlage für diese Arbeit verwendet [9].

2.3.1 Typen der AAS

Analog zu den in Abschnitt 2.1 definierten Abgrenzungen, wurden für die AAS drei Typen, abhängig vom Grad der Integration, definiert [9], [14], [15]:

- **Typ 1 - Passive AAS:** Die AAS wird beispielsweise im AASX-Format als Datei serialisiert ohne aktiv Daten auszutauschen.
- **Typ 2 - Reactive AAS:** Per Application Programming Interfaces (APIs) kann auf die AAS zugegriffen werden, um Daten auszulesen oder Daten des Assets zu speichern.
- **Typ 3 - Proactive AAS:** Die AAS kann aktiv mit dem physischen Asset und anderen AASs über eine Industrie 4.0-konforme Sprache interagieren.

2.3.2 Typ- und Instanz-AAS

AASs werden analog zu dem in Abschnitt 2.2.2 definierten Lifecycle grundsätzlich in zwei Kategorien unterteilt: *Typ-AAS* und *Instanz-AAS*.

Die *Typ-AAS* beschreibt die Struktur und die Eigenschaften eines bestimmten Typs von Assets und wird typischerweise in der Planungsphase eines Assets erstellt. Sie dient als Vorlage für die spätere *Instanz-AAS*, die spezifische Informationen zu einem einzelnen Asset enthält und während des gesamten Lebenszyklus des Assets aktualisiert werden kann [9]. Analog zu Klassen und Objekten in der objektorientierten Programmierung, kann eine *Typ-AAS* als Klasse betrachtet werden, während eine *Instanz-AAS* einem Objekt dieser Klasse entspricht.

2.3.3 Submodels

Die Daten innerhalb der **AAS** sind in sogenannten *Submodels* organisiert. Diese strukturierten Datencontainer enthalten spezifische Informationen zu einem Asset. Die **IDTA** hat bisher knapp 50 verschiedene Submodel-Templates veröffentlicht, die für bestimmte Anwendungsfälle genutzt werden können, weitere sind aktuell in Prüfung. Dabei kann es sich um einen **EU DPP**, technische Daten oder die Klimabilanz eines Produkts handeln [16].

Zusätzlich zu den standardisierten Submodel-Templates können auch individuelle sog. *Custom Submodels* erstellt werden, um spezifische Anforderungen zu erfüllen.

2.3.4 SubmodelElements

Auch wenn die Strukturen der Submodels sehr unterschiedlich sein können, sind alle in einer hierarchischen Baumstruktur aufgebaut, die auf die gleichen Bausteine – die sogenannten **SubmodelElements (SMEs)** – zurückgreift. Im Rahmen dieser Arbeit wird der Fokus auf folgende **SMEs** gelegt [9]:

- **Property**: Simple Datenelement (Zahl, Text, Bool).
- **MultiLanguageProperty (MLP)**: Property mit Sprachvarianten.
- **SubmodelElementCollection (SMC)**: Ungeordnete Sammlung von SubmodelElements.
- **SubmodelElementList (SML)**: Geordnete Sammlung von SubmodelElements.

2.3.5 Qualifiers

Für jedes **SME** (oder auch das gesamte Submodel) können Metainformationen in Form von sogenannten *Qualifiers* hinterlegt werden. Diese Qualifiers sind zusätzliche Informationen, die das **SME** näher beschreiben und dessen Verwendung spezifizieren. Das Metamodell [9] gibt vor, dass sie beispielsweise Informationen über die Einheit eines Wertes, die Gültigkeit eines Datums oder

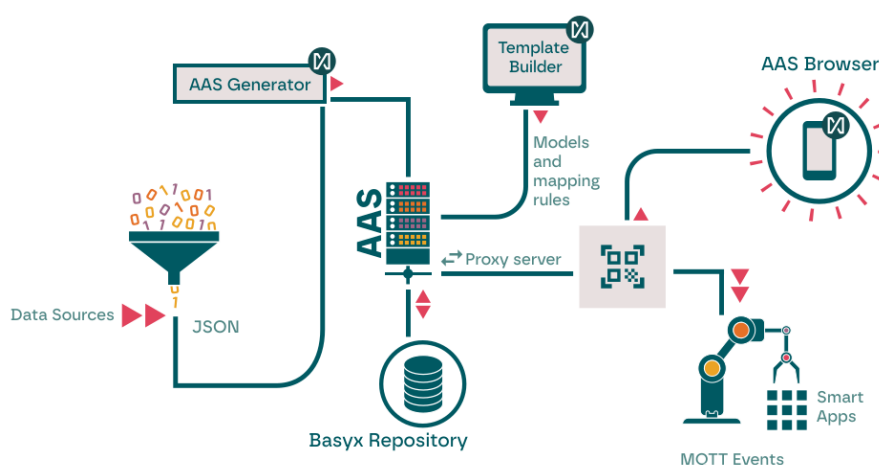
auch die Quelle der Information enthalten können. Letztere, die sogenannten *Template Qualifiers*, sollen in Kapitel 5 verwendet werden, um die Regeln der Rules-Engine zu definieren.

2.4 Eclipse Mnestix-Projekt

Eclipse Mnestix ist ein von der Firma XITASO entwickeltes Open-Source-Projekt, das verschiedene Funktionen für die Verwaltung und Veranschaulichung von AASs bietet [17]. Es besitzt eine webbasierte Oberfläche, um AASs zu erstellen und zu visualisieren. Das *Eclipse Mnestix*-Ökosystem besteht aus mehreren Modulen, die in Abbildung 2.3 veranschaulicht werden.

2.4.1 AAS-Browser

Der AAS-Browser (zuvor *Mnestix-Browser*) stellt eine grafische Benutzeroberfläche zur Verfügung, um AASs zu visualisieren. Er ermöglicht es, die Struktur der AASs zu erkunden und die enthaltenen Submodels und SMEs anzuzeigen, sowie miteinander zu vergleichen. Der AAS-Browser nutzt die *React.JS*-Bibliothek gemeinsam mit dem *Next.JS*-Framework, um eine reaktive und interaktive Benutzeroberfläche zu bieten [17].



2.3 Abbildung: Aufbau des Mnestix-Projekts [17]

2.4.2 Eclipse BaSyx

Die grundlegende Infrastruktur für das *Mnestix*-Ökosystem wird von *Eclipse BaSyx* bereitgestellt. Diese Open-Source-Software wurde zum großen Teil vom *Fraunhofer-Institut für Experimentelles Software Engineering (IESE)* entwickelt und bietet eine Implementierung der AAS-Spezifikation.

Das *BaSyx-Repository* stellt eine [Representational State Transfer \(REST\)-API](#) zur Verfügung, die es ermöglicht, auf [AASs](#) und Submodels zuzugreifen und diese zu bearbeiten. Außerdem sorgen Services wie das [AAS-Registry](#), das [AAS-Discovery](#) und die [AAS-Environment](#) gemeinsam mit einer *MongoDB*-Datenbank für die Speicherung und Verwaltung der [AASs](#). [18]

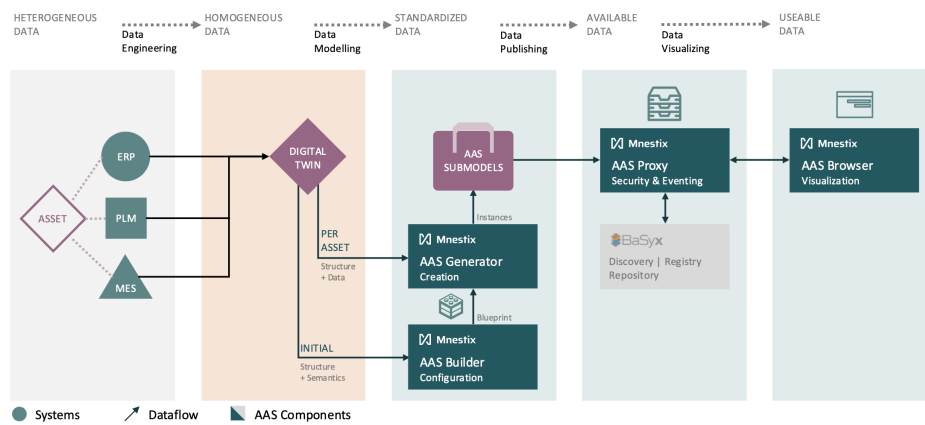
2.4.3 Template Builder

Der *Template Builder* bietet die Möglichkeit Vorlagen zur Erstellung von Submodels zu definieren. Diese Vorlagen können auf ausgewählten [IDTA](#)-Submodel-Templates basieren oder individuell erstellt werden. Es können mit Hilfe von [Qualifiern](#) Regeln für die Erstellung von [SMEs](#) bzw. die Befüllung dieser mit Daten definiert werden. Es können für Property-Elemente [JavaScript Object Notation \(JSON\)](#)-Pfade festgelegt werden, die auf den gewünschten Wert im zu verarbeitenden Datensatz zeigen [17].

2.4.4 AAS-Generator

Der [AAS-Generator](#) (zuvor *DataIngest*) kann nun mit Hilfe der im *Template Builder* definierten Regeln Submodels generieren.

Der gesamte Dataflow wird in [Abbildung 2.4](#) dargestellt. Zunächst werden alle für ein Asset relevanten Daten mittels der Erstellung passender Middleware pro Anwendungsfall, dem sog. Data-Engineering, in eine homogene Struktur gebracht. Diese werden dann als [JSON](#)-Objekt gemeinsam mit der [Identifier \(ID\)](#) des Templates und der [ID](#) der [AAS](#), zu der das Submodel hinzugefügt werden soll, per [REST-API](#)-Aufruf dem [AAS-Generator](#) übergeben. Der [AAS-Generator](#) erstellt dann ein Submodel, das den im Template definierten Regeln entspricht und die konfigurierten Datentransfers aus dem [JSON](#)-Datensatz durchführt. Das Submodel wird der gewünschten [AAS](#) hinzugefügt und anschließend im *Eclipse BaSyx Repository* abgelegt [17], [19].



2.4 Abbildung: Dataflow in Mnestix [19]

3 Related Work

In diesem Kapitel wird eine Übersicht über die relevante Literatur und bestehende Ansätze, automatisiert AASs zu erstellen, gegeben. Zunächst soll der aktuelle Stand der Fachliteratur zu dem Thema beleuchtet werden. Anschließend wird in einer kurzen Marktrecherche ein Überblick über Open-Source-Software gegeben, die in diesem Kontext verwendet werden kann.

3.1 Literaturanalyse

Für dieses Kapitel wurde eine systematische Recherche in verschiedenen wissenschaftlichen Datenbanken durchgeführt. Dabei wurde nach Quellen mit Kombinationen der Schlagwörter „Asset Administration Shell“, „Generation“, „Creation“ und „Automation“ gesucht. Die Suche wurde auf Veröffentlichungen der letzten fünf Jahre beschränkt, um ausschließlich aktuelle Entwicklungen zu berücksichtigen.

Die Recherche ergibt nur wenige Quellen, die sich explizit mit der automatisierten Erstellung von AASs beschäftigen. Die meisten Arbeiten konzentrieren sich auf die Definition und Implementierung von AASs im Allgemeinen oder auf spezifische Anwendungsfälle.

Das Paper *A Digital Twin Description Framework and Its Mapping to Asset Administration Shell* [20] aus dem Jahr 2023 mapped die Fähigkeiten der AAS auf 14 selbst festgelegte Merkmale von Digital Twins. Dabei unterstützt die AAS 12 der 14 Merkmale explizit oder implizit. Das Paper zeigt in einer Fallstudie, wie eine reaktive AAS für eine Bohrmaschine erstellt werden kann, die Sensordaten in Echtzeit erfasst und verarbeitet. Die Autor:innen greifen die Definitionen von *Kritzinger et al.* [4] (vgl. Abschnitt 2.1) auf und kategorisieren diesen Aufbau als *Digital Shadow*. Jedoch wird in diesem Paper nicht beschrieben, wie auf technischer Ebene die Daten in die AAS überführt werden.

In einem Research-Paper aus dem Jahr 2023 [21] wird ein semi-automatischer Ansatz zur Erstellung von AASs vorgestellt, der auf der Verwendung von Künstliche Intelligenz (KI)-Methoden basiert. Zunächst werden aus heterogen strukturierte Daten verschiedener Quellen jeweils einzelne AASs erstellt. Bei diesen Daten kann es sich um strukturierte Daten wie beispielsweise XML-Dateien, aber auch unstrukturierte Daten wie Portable Document Format (PDF)-Textdokumente handeln. Für letztere wird Natural Language Processing (NLP) verwendet, um die relevanten Informationen zu extrahieren. Zum Schluss werden die erstellten AASs zusammengeführt und aufgetretene Fehler manuell korrigiert. Dieser Ansatz kann die Erstellung neuer unterschiedlich strukturierter AASs

stark beschleunigen, jedoch nicht vollständig automatisieren. Obwohl die aktuelle Entwicklung von KI-Methoden in Zukunft eine vollständige Automatisierung ermöglichen könnte, wird in dieser Arbeit ein deterministischer Ansatz verfolgt, der auf die Verwendung von KI verzichtet.

In *Semantic Integration Patterns for Industry 4.0* [22] werden [Semantic Integration Patterns \(SIPs\)](#) vorgestellt, die es ermöglichen, verschiedene Kommunikationsprotokolle semantisch miteinander zu verknüpfen. Der Aufwand, Daten in andere Formate umzuwandeln, soll mit diesem Ansatz verringert werden. Prinzipien wie „Vererbung“, „Typ/Instanz-Beziehungen“ und „verpflichtende [SemanticIDs](#)“ werden genutzt, um den Integrationsaufwand zu reduzieren. Es wurden Bausteine entwickelt, die gemeinsam eine Middleware zwischen verschiedenen Assets und Anwendungen bilden. Über diese Middleware können Assets und Anwendungen Informationen und Funktionen bereitstellen und nutzen. Jede Information und jede Funktion hat eine [SemanticID](#), die es zum Beispiel einer Komponente ermöglicht, einen Wartungsauftrag zu erstellen, ohne zu wissen welches Wartungssystem verwendet wird.

Guntner et al. [23] implementieren das Konzept konkret. Die Middleware soll einen eingehenden Wartungsauftrag in einer [AAS](#) abbilden. Zunächst wird der Wartungsauftrag gemeinsam mit seinem semantischen Identifikator von der Middleware entgegen genommen. Der Identifikator ist in einem *Semantic-Lookup-Service* hinterlegt. Hier wird die [International Electrotechnical Commission \(IEC\)](#) 61360-Datenbank verwendet [24]. Diese liefert die Struktur des Wartungsauftrages in Form von Klassen und Attributen verschiedener Datentypen. Im Anschluss werden diese Elemente in die [AAS](#) überführt. Diese Implementierung bietet nicht die Funktion, Regeln komplett frei manuell zu definieren, allerdings sollten die meisten Anwendungsfälle mit Katalogen wie dem [IEC61360](#) abgebildet sein.

Ein Artikel aus 2021 [25] baut eine virtuelle Produktionslinie auf, in der verschiedene Maschinen über ihre [AASs](#) miteinander kommunizieren. Dabei wird ein sogenannter *Configuration-Wizard* entwickelt, der es ermöglicht, über eine [Graphical User Interface \(GUI\)](#) auf das [Open Platform Communications Unified Architecture \(OPC UA\) Software Development Kit \(SDK\)](#) zuzugreifen und anschließend [AASs](#) zu erstellen. [OPC UA](#) ist ein Standard für den Austausch von semantisch beschriebenen Maschinendaten [26]. In diesen *Configuration-Wizard* werden die Daten manuell eingetragen. Die Autor:innen bezeichnen diesen Prozess als „automatisiert“, da die Daten nicht mehr von Hand in eine [XML](#)-Datei geschrieben werden müssen. Dies reduziert die Dauer der Erstellung von [AASs](#) und garantiert eine korrekte Syntax nach dem [AAS](#)-Metamodell, erreicht jedoch keine vollständige Automatisierung, wie sie in dieser Arbeit angestrebt wird.

Ein weiterer Ansatz [27] beschäftigt sich mit der Frage, wie das [AAS](#)-Metamodell automatisiert in verschiedene Schemata-Sprachen wie [JSON](#)-Schema, [XML Schema Definition \(XSD\)](#) oder [Resource Description Framework \(RDF\)](#) erstellt werden kann. Die Spezifikation wird zunächst in Python definiert und anschließend in die jeweiligen Schemata-Sprachen überführt. Auch wenn es sich hierbei nicht um die Erstellung von konkreten [AASs](#) handelt, ist der Ansatz interessant, da Regeln innerhalb einer Baumstruktur ausgewertet werden, um die verschiedenen Schemata zu generieren.

Man findet in der Literatur verschiedene Ansätze, wie einzelne konkrete Datenformaten in AAS überführt werden können.

In *AAS for PLC Representation Based on IEC 61131-3* [28] sollen Programme für **Programmable Logic Controllers (PLCs)** nach der IEC 61131-3 Norm in AAS-Submodels abgebildet werden. Hierfür wird ein AAS-Submodel erstellt, das die Struktur und Funktionalität der PLC-Programme repräsentiert. Allerdings beschreibt die Arbeit nur, was abgebildet werden soll, nicht, wie die Abbildung automatisiert erfolgen kann. Im Ausblick erwähnen die Autor:innen, dass dies ein möglicher nächster Schritt sein könnte.

Schmidt et al. [29] beschäftigen sich mit der Abbildung von **Digital Twin Definition Language (DTDl)** Modellen in AASs. DTDl ist ein von Microsoft entwickeltes Format zur Beschreibung von digitalen Zwillingen, welches von den *Azure Digital Twins* verwendet wird [30]. Es wurde ein Prototyp entwickelt, der DTDl-Modelle in AASs überführt. Allerdings endet die Implementierung bei simplen Datentypen wie Integer und String. Komplexere Datentypen wie Arrays oder Objekte werden nicht unterstützt.

In einem anderen Werk [31] sollen **OPC UA** Daten in AAS-Daten umgewandelt werden. Es wird eine Architektur vorgestellt, die es ermöglicht, Daten mit Hilfe der *Eclipse BaSyx DataBridge* (vgl. Abschnitt 3.2.2) in AAS-Submodels zu überführen. Es wird hierbei ein rekursiver Ansatz genutzt. Allerdings ist die Implementierung nur auf **OPC UA** Daten beschränkt und von der *Eclipse BaSyx* Umgebung stark abhängig.

Ein Paper aus 2020 [32] beschreibt, wie aus **Automation Modelling Language (AML)**-Dateien regelbasiert AASs erstellt werden können. AML ist ein offen standardisiertes XML-basiertes Format zur Beschreibung von industriellen Anlagen und deren Komponenten [33]. Es wird ein Prototyp entwickelt, der *AutomationML*-Dateien einer Messzelle in AAS-Submodels überführt. Hierfür werden einmalig Mapping-Regeln definiert, die beschreiben, wie die Elemente aus der AML-Datei in AAS-Elemente überführt werden. Allerdings ist die Implementierung nur auf AML-Dateien beschränkt.

Obwohl die zuletzt genannten Werke sich bereits mit der automatisierten Erstellung von AASs beschäftigen, sind sie meist auf ein konkretes Datenformat beschränkt. In dieser Arbeit soll ein generischer Ansatz verfolgt werden, der es ermöglicht, verschieden strukturierte Daten in AASs zu überführen. Trotzdem können bestimmte Konzepte und Ansätze aus den genannten Werken übernommen und erweitert werden.

3.2 Marktanalyse

Als Ausgangspunkt der Marktanalyse wurde eine Studie aus dem Jahr 2023 [15] analysiert, die verschiedene Open-Source-Implementierungen von AASs untersucht. In dieser Studie wurde der Fokus auf vier Projekte gelegt.

Die vier gefundenen Projekte sind:

1. **AASX Server** ([GitHub Repository](#))

2. **Eclipse BaSyx** ([GitHub Repository](#))
3. **FA³ST Service** ([GitHub Repository](#))
4. **NOVAAS** ([GitLab Repository](#))

Im Folgenden werden die vier Projekte kurz beschrieben und die Funktionalität zur automatisierten Erstellung von **AASs** bewertet.

3.2.1 AASX Server

Der **AASX** Server ist eine Companion-App für den beliebten **AASX** Package Explorer, welcher Funktionen zur Visualisierung und Bearbeitung von **AASs** bietet [34], [35]. Der **AASX** Server erweitert diese Funktionalität um Endpunkte für **REST**, **OPC UA** und **Message Queuing Telemetry Transport (MQTT)**, die es ermöglichen, **AASs** zu erstellen, zu lesen, zu aktualisieren und zu löschen. Es können Änderungen von **OPC UA** basierten Assets abonniert werden, um simple Werte in den **AASs** automatisch zu aktualisieren. Diese Funktionalität lässt sich per **AAS**-Qualifier definieren. Allerdings wird vorausgesetzt, dass die Node-IDs in **OPC UA** nach einer bestimmten Konvention benannt sein müssen, was im produktiven Einsatz meist nicht möglich ist [15].

Das Projekt ist seit 2019 aktiv und befindet sich noch in der Alpha-Phase. Neue Versionen werden eher unregelmäßig veröffentlicht. Die neueste Version wurde zum Zeitpunkt der Recherche am 8.05.2024 veröffentlicht [34].

3.2.2 Eclipse BaSyx

Da der Mnestix-Browser mit Eclipse BaSyx Komponenten arbeitet, wurde in Abschnitt 2.4.2 bereits ein grober Überblick über das Projekt gegeben. Das Eclipse BaSyx Projekt des Fraunhofer-**IESE** ist eine weit etablierte Sammlung von Tools und Bibliotheken, die die **AAS** implementieren. Es gibt **SDKs** für verschiedene Programmiersprachen wie Java, C# und Python [36]. In Modulen wie dem **AAS Server** und der **AAS Registry** werden verschiedene Funktionalitäten zur Verwaltung und Kommunikation von **AASs** bereitgestellt [18].

Für diese Arbeit besonders relevant ist die *DataBridge*-Komponente, die es ermöglicht, Daten aus verschiedenen Quellen in **AASs** zu überführen [37]. In einem Blogartikel des Instituts [38] wird beschrieben, wie die *DataBridge* genutzt werden kann, um Daten in **AASs** zu überführen. Das Tool kann Daten zyklisch in die **AAS** laden oder on-demand abfragen. Zudem können Ausdrücke in der Abfragesprache *JSONata* verwendet werden, um die Daten zu transformieren. *JSONata* bietet verschiedene Funktionen zum Auslesen, Transformieren und Aggregieren von Daten [39]. Allerdings kann jede Mapping-Regel nur auf ein einzelnes **SME** angewendet werden. Es ist nicht möglich, komplexere Regeln zu definieren, die mehrere Elemente gleichzeitig befüllen.

An den Eclipse BaSyx Projekten wird seit mehreren Jahren aktiv gearbeitet. Für die *DataBridge* wurde zum Stand der Recherche zuletzt im Januar 2025 eine Änderung vorgenommen [40].

3.2.3 FA³ST Service

Ähnlich wie das Eclipse BaSyx Ökosystem sollen im FA³ST Projekt verschiedene Module mit verschiedenen Funktionalitäten bereitgestellt werden [41]. Das Projekt wird vom Fraunhofer-Institut für Optronik, Systemtechnik und Bildauswertung (IOSB) entwickelt. Das Modul *FA³ST Mapper* soll es ermöglichen, Daten aus verschiedenen Quellen in AASs zu überführen. Auf der Webseite des IOSB wird beschrieben, dass das Lesen und Schreiben von Werten, das Verarbeiten von Funktionsaufrufen und das Abonnieren von Ereignissen unterstützt wird [41]. Allerdings konnte weder der Quellcode noch detaillierte Dokumentation über diese Komponente öffentlich gefunden werden, um näher auf das Tool einzugehen.

Für den sog. *FA³ST Service* als zentrales Bestandteil des Projekts wurde zuletzt Ende 2024 eine neue Version veröffentlicht, allerdings zeigt die Commit-Historie, dass aktuell aktiv an dem Projekt gearbeitet wird [42].

3.2.4 NOVAAS

Das NOVAAS-Projekt ist eine Referenz-Implementierung der AAS, die von der Nova School of Science and Technology in Portugal entwickelt wird [43], [44]. Als Grundlage wird das JavaScript-basierte Low-Code-Tool *Node-RED* verwendet. Node-RED ist eine visuelle Programmierungsumgebung, die es ermöglicht, ereignisgesteuerte Anwendungen durch das Verbinden von vorgefertigten Bausteinen zu erstellen [45]. NOVAAS zielt vor Allem darauf ab, eine Brücke zwischen der Information Technology (IT)-Welt und Operational Technology (OT)-Welt zu schaffen. Dabei soll die AAS-Spezifikation in etablierten Web-Technologien wie JavaScript und REST implementiert werden.

Im Gegensatz zu den bisher vorgestellten Projekten bietet NOVAAS weniger Funktionalitäten zur Verwaltung von AASs [15]. Mit Hilfe eines *Node-RED-Flow* können Daten per Hypertext Transfer Protocol (HTTP) oder OPC UA in AASs überführt werden. In dem vorgegebenen Beispiel werden nur simple Datentypen in AAS-Properties überführt. [46] Allerdings können komplexere Mapping-Regeln durch das Erstellen eigener Node-RED-Flows implementiert werden, sofern genügend Kenntnisse über Node-RED vorhanden sind.

An dem Projekt wird seit 2020 moderat aktiv gearbeitet. In den letzten Monaten wurden einige kleinere Änderungen vorgenommen [43].

3.3 Fazit

Die Literatur- und Marktanalyse zeigt, dass es bereits einige Ansätze und Tools gibt, die sich mit der automatisierten Erstellung von AASs beschäftigen. Allerdings sind viele dieser Ansätze entweder auf spezifische Datenformate beschränkt oder bieten nur begrenzte Funktionalitäten zur Definition komplexerer Mapping-Regeln.

Die Analyse hebt das Zusammenführen von heterogen strukturierten Daten als eine der wichtigsten Herausforderungen der Industrie 4.0 hervor. Es werden viele Lösungen benötigt, die es ermöglichen, Daten aus verschiedenen Quellen in AASs zu überführen. Dabei ist es wichtig, dass die Lösungen flexibel und erweiterbar sind, um den unterschiedlichen Anforderungen gerecht zu werden.

4 Anforderungsanalyse

In diesem Kapitel soll analysiert werden, welche Anforderungen an die Rules-Engine gestellt werden. Dieses Kapitel wird nach der *ISO/IEC/IEEE 29148 Systems and Software Requirements Specification (SRS)* [47], [48] aufgebaut. Zusätzlich werden in Abschnitt 4.13 **Acceptance Criteria (ACs)** (Akzeptanzkriterien) definiert, die die Anforderungen verifizierbar machen. Zuletzt werden noch in Abschnitt 4.14 die Anforderungen mit den **Use Cases (UCs)** und Qualitätszielen verknüpft.

4.1 Zweck und Scope

Ziel dieses Kapitels ist die vollständige, eindeutige und überprüfbare Spezifikation der Anforderungen an die *Rules-Engine* zur automatisierten Erstellung von **AAS**-Submodels aus strukturierten Eingangsdaten. Im Scope dieser Spezifikation liegen:

- die fachliche Beschreibung der zu unterstützenden Regeltypen und Prozesse zur Submodel-Erzeugung,
- die Schnittstellen zum bestehenden *Eclipse Mnestix*-Ökosystem (insb. **AAS**-Generator) und
- die Qualitätsanforderungen (Betrieb, Performance, Wartbarkeit etc.).

Außerhalb des Scopes liegen algorithmische Implementierungsdetails, konkrete UI-Designs und ausführliche Markt-/Technologiediskussionen (vgl. Kapitel 3).

4.2 Begriffe und Referenzen

Normative Schlüsselwörter: In Anlehnung an [47] bedeuten: **MUSS** (verbindlich), **SOLL** (stark empfohlen/Normalfall), **DARF / KANN** (optional/Erlaubnis). **WIRD** beschreibt Kontext, nicht Normatives.

AAS-Terminologie: Die Begriffe *Submodel*, **SME**, **SMC**, **SML** und *Qualifier* werden konsistent gemäß dem **AAS**-Metamodell [9] verwendet. *Submodel-Template* bezeichnet ein Vorlagen-Submodel, dessen Qualifier Regelinformationen enthalten (Details und Beispiele in Kapitel 5).

Anforderungs-IDs und Attribute: Jede Anforderung besitzt *Typ* (FR/NFR/CON), *ID*, *Titel*, *Beschreibung* und *Priorität* (1–3). Die Prioritätsskala ist: 1 = hoch, 2 = mittel, 3 = niedrig.

4.3 Systemkontext

Die Rules-Engine erweitert den bestehenden *AAS-Generator* des *Eclipse Mnestix*-Ökosystems um weitere regelbasierte Datenabbildung in Submodels. Es werden strukturierte *JSON*-Daten per *REST-API* an die Rules-Engine übergeben, ein neues Submodel erstellt und anschließend in einer vorgegebenen *AAS* abgelegt. Die Architektur und Komponenten von *Mnestix* werden in Kapitel 2 beschrieben, die logische Regelrepräsentation in Kapitel 5.

4.4 Stakeholder

Orientiert an den bestehenden Personas von *Eclipse Mnestix* [49] werden für verschiedene Nutzerklassen verschiedene Interessen und Ziele definiert. Diese werden in Tabelle 4.1 aufgeschlüsselt.

Tabelle 4.1: Stakeholderübersicht	
Stakeholder (Persona)	Interessen/Ziele
Ana Admin (Mid-level Admin)	Intuitive Template-Verwaltung; Übersicht über Templates/Versionen; robuste Fehlermeldungen.
Pablo Biz (Executive)	Nutzen/Return on Investment, Zukunftssicherheit; minimale Betriebskosten durch Automatisierung.
Noah Developer (AAS Provider)	<i>API</i> -Stabilität; Beispiel-Templates; Testdaten; Erweiterbarkeit.
Lewis Operator (AAS Consumer)	Verlässliche, nachvollziehbare Submodel-Erzeugung; kein manueller Nacharbeitsbedarf.

4.5 Betriebsumgebung, Annahmen und Abhängigkeiten

Betriebsumgebung: Die Rules-Engine läuft integriert im *Mnestix-Backend* (*AAS*-Generator) und interagiert mit den bereits vorhandenen *AAS*-Services (Registry/Environment). Die *GUI* wird im *AAS Browser* bereitgestellt (vgl. Abschnitt 2.4).

Annahmen:

- (A1) Eingangsdaten liegen als valide, strukturierte *JSON*-Dokumente vor. Die Homogenisierung erfolgt vorgelagert über Verfahren wie *Extract, Transform, Load* (ETL) oder Quellen-spezifische Adapter.

- (A2) Submodel-Templates enthalten die für das Mapping benötigten Qualifier gemäß der in Kapitel 5 definierten Form.
- (A3) Relevante Mnestix-Komponenten bleiben [API](#)-kompatibel. Anpassungen an Altsystemen sind nicht Teil dieser Arbeit.
- (A4) Zukünftige Versionen des [AAS](#)-Metamodells [9] und Änderungen an anderen Mnestix-Komponenten führen nicht zu Breaking-Changes, die die Rules-Engine betreffen.

Abhängigkeiten:

- (D1) *Eclipse Mnestix* (Backend/Frontend) [50] - stabile [APIs](#) und Betrieb.
- (D2) [AAS](#)-Metamodell v3.* [9] - keine Breaking-Changes.

4.6 Schnittstellen und Datenquellen

Die in diesem Abschnitt beschriebenen Schnittstellen und Datenquellen sind als Anforderungen an die Umgebung und nicht das System selbst zu verstehen.

Externe Schnittstellen: Es **MUSS** eine [REST-API](#) bereitstehen zur Template-Verwaltung und zur Datenübergabe bei einer Submodel-Erzeugung.

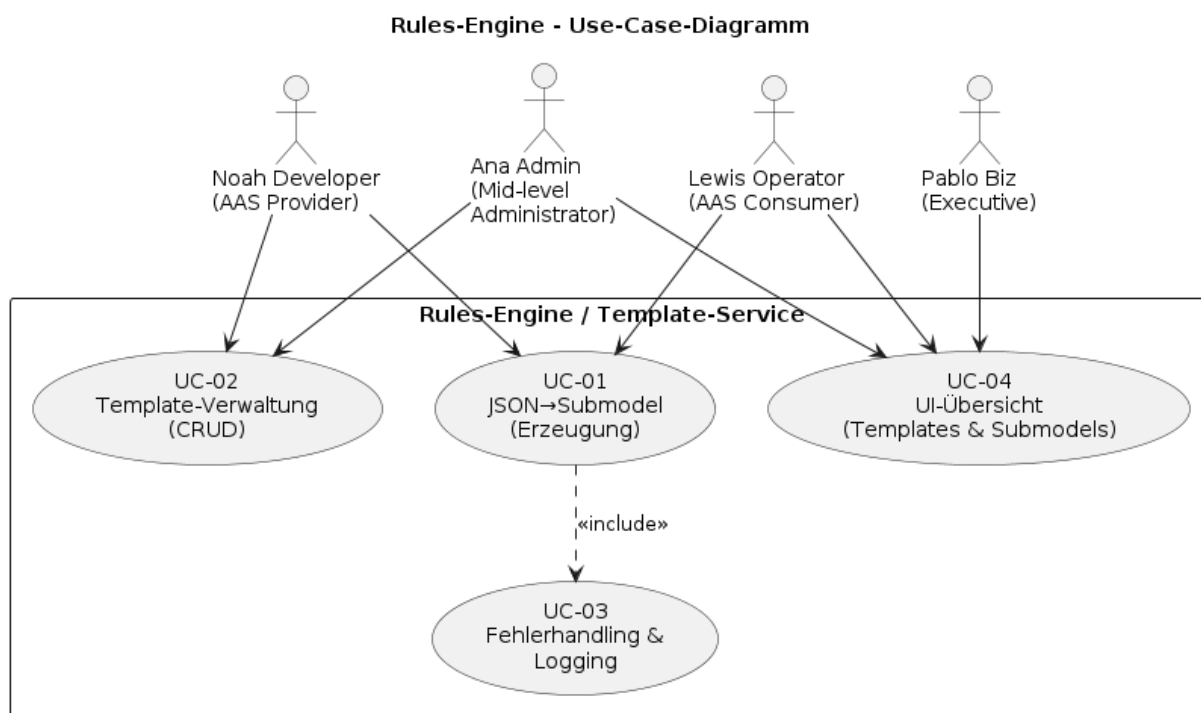
Datenquellen: Es **MÜSSEN** strukturierte [JSON](#)-Daten eingegeben werden. Die Größe und Form **KÖNNEN** pro Use-Case variieren. [AASX/XML](#)-Serialisierungen oder unstrukturierte Quellen ([PDF](#)/Text) sind **nicht** Bestandteil dieser Anforderungen.

4.7 Use-Cases

Die folgenden [UCs](#) werden für die Rules-Engine definiert:

- UC-01 JSON→Submodel (Erzeugung):* Ein Nutzer sendet über die [REST-API](#) ein [JSON](#)-Objekt zusammen mit der [ID](#) eines Submodel-Templates und der Ziel-[AAS-ID](#). Die Rules-Engine verarbeitet die Daten gemäß den Template-Regeln und erzeugt ein neues Submodel, das an die angegebene [AAS](#) angehängt wird.
- UC-02 Template-Verwaltung:* Ein Nutzer kann über die [REST-API](#) Submodel-Templates erstellen, lesen, aktualisieren und löschen. Die Templates werden in der Mnestix-Umgebung gespeichert und sind für die Submodel-Erzeugung verfügbar.
- UC-03 Fehlerhandling & Logging:* Ein Nutzer kann falsch strukturierte Daten über die [API](#) senden. Die Rules-Engine validiert die Eingaben, gibt strukturierte Fehlermeldungen zurück und loggt Fehler für Diagnosezwecke.
- UC-04 UI-Übersicht:* Ein Nutzer kann im Mnestix-Frontend verfügbare Templates und generierte Submodels einsehen.

Die UCs werden in Abbildung 4.1 mit den zuvor definierten Stakeholdern verknüpft. Der Fokus dieser Arbeit liegt auf UC-01 und UC-03, während UC-02 und UC-04 sich auf andere Komponenten des *Mnestix*-Ökosystems beziehen.



4.1 Abbildung: Use-Case-Übersicht der Rules-Engine

4.8 Qualitätsziele

Die Qualitätsziele orientieren sich an der *ISO/IEC25010*-Norm [51] und wurden in einem Qualitäts-Workshop mit Stakeholdern des *Mnestix*-Projekts [52] weiter verfeinert.

- Q-01 Performance:* Große Submodels lassen sich in einem angemessenen Zeitrahmen erstellen.
- Q-02 Wartbarkeit:* Mit hinreichender Dokumentation können Wartungen wie z.B. Versionsupdates der verwendeten Bibliotheken einfach durchgeführt werden.
- Q-03 Erweiterbarkeit:* Es können ohne große Änderungen am bestehenden System neue Regeln hinzugefügt werden.

4.9 Funktionale Anforderungen

Feature F1: Regelbasierte Abbildung in Submodels

- FR-01 (Prio 1) Regeltypen:* Das System **MUSS** folgende Regeltypen unterstützen: (a) *Default-Werte*, (b) *Pfadregeln* (1:1-Mapping von Datenpfaden), (c) *Schleifenregeln* (Iteration über Listen/Collections, ggf. inkl. Indexersetzung), (d) *Filter-Regeln* (Filter auf Basis von Datenwerten).
- FR-02 (Prio 1) Optionale und Pflichtfelder:* Sind Felder im Template mit einer Regel versehen, die im Eingabedatensatz nicht erfüllt werden kann (z. B. Pfad nicht vorhanden), **MÜSSEN** optionale Felder übersprungen werden und bei verpflichtenden Feldern **MUSS** die Submodel-Erzeugung mit einer strukturierten Fehlermeldung fehlschlagen.
- FR-03 (Prio 1) Regel-Ablage:* Regeln **MÜSSEN** **AAS**-konform als Qualifier in Submodel-Templates abgelegt und mittransportiert werden.
- FR-04 (Prio 1) SME-Unterstützung:* Das System **MUSS** Mapping für *Property*, **MLP**, **SMC** und **SML** unterstützen (keine Einschränkung auf nur *Property*).
- FR-05 (Prio 1) Typ → Instanz-Erzeugung:* Aus Typ-Submodels **MÜSSEN** Instanz-Submodels generiert werden; Template-Qualifier **MÜSSEN** entfernt oder in geeignete Concept-Qualifier überführt werden.

Feature F2: Template- und Daten-API

- FR-06 (Prio 1) Template-Verwaltung:* Es **MUSS** **REST**-Endpunkte für die **Create, Read, Update, Delete (CRUD)**-Operationen von Submodel-Templates geben.
- FR-07 (Prio 1) AAS-Generator:* Es **MUSS** einen **REST**-Endpunkt geben, der (a) Template-**ID**, (b) Eingangsdaten (**JSON**) und (c) Ziel-**AAS-ID** entgegennimmt und daraus ein Submodel erzeugt.

Feature F3: Validierung, Fehlerbehandlung und Protokollierung

- FR-08 (Prio 1) Validierungsfehler:* Tritt bei der Submodel-Erzeugung ein Validierungsfehler auf (z. B. fehlender Pfad, indizierte Liste außerhalb des Bereichs), **MUSS** eine strukturierte Fehlermeldung mit Template-/Regel-**ID**, betroffener **SME**-Referenz und Fehlerart über die **API** zurückgegeben werden.
- FR-09 (Prio 2) Logging:* Tritt bei der Submodel-Erzeugung ein Validierungsfehler auf (z. B. fehlender Pfad, indizierte Liste außerhalb des Bereichs), **MUSS** eine strukturierte Fehlermeldung mit Template-/Regel-**ID**, betroffener **SME**-Referenz und Fehlerart auf **ERROR**-Ebene geloggt werden.

Feature F4: GUI-Unterstützung

FR-10 (Prio 2) UI-Übersichten: Die **GUI MUSS** Übersichten über verfügbare Templates und generierte Submodels bereitstellen; Visualisierung von Templates/Strukturen **SOLL** unterstützt werden.

FR-11 (Prio 3) UI-Diagnose: Die **GUI SOLL** den Erstellungsablauf und aufgetretene Validierungsfehler visualisieren.

4.10 Nicht-funktionale Anforderungen

NFR-01 (Prio 1) Konformität: Erzeugte Submodels **MÜSSEN** dem **AAS**-Metamodell mit der Major Version 3 [9] entsprechen.

NFR-02 (Prio 1) Erweiterbarkeit: Neue Regeltypen **MÜSSEN** modular implementierbar sein, ohne bestehende zu ändern.

NFR-03 (Prio 2) Performance: Die Erzeugung eines Submodels mit 10 000 Listenelementen **DARF** ≤ 10 s dauern (Messung auf Referenzhardware; Datensatz und Messverfahren in Kapitel 7).

NFR-04 (Prio 2) Testbarkeit: Unit-Testabdeckung **SOLL** ≥ 80 % betragen.

NFR-05 (Prio 2) Sicherheit/Robustheit: Eingabedaten **MÜSSEN** validiert werden; Laufzeitfehler **MÜSSEN** sicher vom System aufgefangen werden.

4.11 Randbedingungen

CON-01 (Prio 1) Ökosystem: Integration in das bestehende *Mnestix*-Ökosystem (**AAS**-Services, Registry/Environment) ist zwingend.

CON-02 (Prio 1) Schnittstellenfokus: Fokus auf **REST-API** (Template-Verwaltung, Data-Ingest); **AASX/XML** sind nicht Gegenstand dieser Arbeit.

CON-03 (Prio 2) Lizenz/Offenheit: Veröffentlichung konsistent zum *Mnestix*-Projekt unter MIT-Lizenz (Eclipse Foundation) [50].

CON-04 (Prio 3) Technologie-Rahmen: Technologiewahl orientiert sich am bestehenden Stack: *.Net 8-Core*-Backend, *NextJS 15*-Frontend

4.12 Verifikationsmethoden

V1 Test: Ausführung gegen definierte Datensätze

V2 Analyse: statische Prüfung/Code-Analyse/Schema-Validierung

V3 Demonstration: UI/Workflow-Demo

V4 Inspektion: Review von Artefakten/Logs.

4.13 Akzeptanzkriterien (ACs)

AC-01 — Pfadmapping korrekt (FR-01) — Methode: Test

Gegeben: Ein Template mit verschiedenen Pfadregeln für einfache Properties und dazu passende Eingangsdaten.

Wenn: Die Submodel-Erzeugung ausgeführt wird.

Dann: Die Werte werden korrekt von den Eingangspfaden zu den Ziel-Properties gemappt; die [API](#) liefert Status 200.

AC-02 — Listen-Mapping korrekt (FR-01, FR-04) — Methode: Test

Gegeben: Eingangsdaten mit verschachtelten Listen mit unterschiedlichen Längen und ein Template mit Listenregeln diese Listen.

Wenn: Die Erzeugung mit Template ausgeführt wird.

Dann: Im erzeugten Submodel sind die verschachtelten Listen mit den korrekten Werten abgebildet; die [API](#) liefert Status 200.

AC-03 — Filterregeln (FR-01) — Methode: Test

Gegeben: Ein Template mit einer Filterregel auf ein [SME](#) und Eingangsdaten, die die definierte Bedingung erfüllen.

Wenn: Die Submodel-Erzeugung ausgeführt wird.

Dann: Das erstellte Submodel enthält diese [SME](#).

AC-04 — Optionales Feld fehlt beim Pfadmapping (FR-02) — Methode: Test

Gegeben: Ein Template mit optionalen Feldern und Eingangsdaten, in denen die zugehörigen Daten fehlen.

Wenn: Die Submodel-Erzeugung ausgeführt wird.

Dann: Fehlende optionale Felder werden übersprungen; das Submodel wird erfolgreich erstellt; die [API](#) liefert Status 200.

AC-05 — Pflichtfeld fehlt beim Pfadmapping (FR-02) — Methode: Test

Gegeben: Ein Template mit verpflichtenden Feldern und Eingangsdaten, in denen die zugehörigen Daten fehlen.

Wenn: Die Submodel-Erzeugung ausgeführt wird.

Dann: Die Erzeugung schlägt mit strukturierter Fehlermeldung fehl; die [API](#) liefert Status 400; die Fehlermeldung enthält Template-[ID](#), [SME](#)-Referenz und fehlenden Pfad.

AC-06 — Regel-Ablage AAS-konform (FR-03) — Methode: Analyse

Gegeben: Ein Submodel-Template mit verschiedenen Regeltypen (Pfad-, Schleifen-, Filter-, Default-Regeln).

Wenn: Das Template mittels [AAS](#)-Schema-Validierung geprüft wird.

Dann: Alle Regel-Informationen sind als gültige Qualifier gemäß [AAS](#)-Metamodell v3.* abgelegt; keine Schema-Validierungsfehler auftreten.

AC-07 — Typ→Instanz-Erzeugung (FR-05) — Methode: Test

Gegeben: Ein Typ-Submodel (kind = Template) mit Template-Qualifiers.

Wenn: Eine Erzeugung für eine Instanz erfolgt.

Dann: Das Resultat hat `kind = Instance`; Template-Qualifier sind entfernt oder korrekt in Concept-Qualifier überführt.

AC-08 — Template-CRUD (FR-06) — Methode: Test

Gegeben: API-Endpunkte für das Lesen, Erstellen, Bearbeiten und Löschen von Templates

Wenn: GET, POST, PUT, DELETE Anfragen an die API

Dann: Aktionen Lesen, Erstellen, Bearbeiten und Löschen von Templates ausgeführt.

AC-09 — Data-Ingest Roundtrip (FR-07) — Methode: Test

Gegeben: Ein gültiges Template und eine Ziel-AAS-ID.

Wenn: Ein POST auf /DataIngest mit Template-ID, AAS-ID und JSON erfolgt.

Dann: Antwort 200 mit Submodel-ID; das erzeugte Submodel ist schema-valide (AAS v3.*).

AC-10 — Strukturierte Fehlermeldung & Logging (FR-08, FR-09) — Methode: Test/Inspektion

Gegeben: Ein Template mit einer Pfadregel auf einen im Eingabe-JSON nicht vorhandenen Datenpfad.

Wenn: Die Erzeugung ausgeführt wird.

Dann: Die API liefert eine strukturierte Fehlermeldung mit Template-/Regel-ID, SME-Referenz und Fehlerart und es existiert ein ERROR-Logeintrag mit denselben Referenzen.

AC-11 — UI-Übersichten (FR-10, FR-11) — Methode: Demonstration

Gegeben: Mindestens ein Template und ein erzeugtes Submodel vorhanden.

Wenn: Die GUI geöffnet und die „Templates“-Ansicht aufgerufen wird.

Dann: Die Liste zeigt ID/Name/Version; ein Klick zeigt den SME-Strukturbaum; optional Fehlerlauf mit visualisierter Fehlermeldung.

AC-12 — Performanceziel (NFR-03) — Methode: Test

Gegeben: Ein Referenzdatensatz mit 10 000 Listenelementen.

Wenn: Die Erzeugung auf Referenzhardware/CI ausgeführt wird.

Dann: Die Gesamtdauer beträgt ≤ 10 s.

AC-13 — Testbarkeit/Wartbarkeit (NFR-04) — Methode: Analyse/Test

Gegeben: Entwicklungsumgebung mit eingerichtetem Code Coverage Tool.

Wenn: Das Code Coverage Tool ausgeführt wird.

Dann: Der Coverage-Bericht zeigt $\geq 80\%$ Line- und Branch-Coverage im Verzeichnis MnestixCore/AasGenerator/*.

AC-14 — Erweiterbarkeit Regeltypen (NFR-02) — Methode: Analyse/Test

Gegeben: Ein neuer Regeltyp „YetAnotherRule“ wird als separate Klasse implementiert und in der Pipeline registriert.

Wenn: Die Testsuite ausgeführt wird.

Dann: Keine Codeänderung an bestehenden Regeltypen notwendig (Open/Closed); alle Regressionstests grün.

4.14 Traceability

Die Nachverfolgbarkeit zwischen **UCs** bzw. Qualitätszielen, Anforderungen und **ACs** wird in Tabelle 4.2 dargestellt.

Use-Case / Qualitätsziel	Anforderungen (FR/NFR/-CON)	Nachweis(e) / AC
UC-1: JSON→Submodel (Erzeugung)	FR-01, FR-02, FR-03, FR-05, FR-07; NFR-01	AC-01, AC-02, AC-03, AC-04, AC-05, AC-06, AC-07, AC-09
UC-2: Template-Verwaltung (CRUD)	FR-06; CON-02	AC-08
UC-3: Fehlerhandling & Logging	FR-08, FR-09, NFR-05	AC-10
UC-4: UI-Übersichten/Diagnose	FR-10, FR-11	AC-11
Q-01: Performance	NFR-03	AC-12
Q-02: Wartbarkeit	NFR-04	AC-13
Q-03: Erweiterbarkeit	NFR-02	AC-14

Tabelle 4.2: Traceability-Matrix

4.15 Zusammenfassung

Die Anforderungen definieren klar die regelbasierte Submodel-Erzeugung, die notwendigen **APIs**, die Validierungs-/Fehlerbehandlung und die Qualitätsziele. Sie sind konsistent in MUSS/-SOLL/DARF formuliert und jeweils verifizierbar.

Die Prioritäten der Anforderungen wurden passend zu dem Fokus auf *UC-01* und *UC-03* gesetzt.

5 Darstellung der Regeln

In diesem Kapitel wird beschrieben, wie die in [AAS-Submodel-Templates](#) hinterlegten *Template-Qualifier* (siehe Abschnitt [2.3.5](#)) zur Darstellung von den in Kapitel 4 geforderten Regeltypen (*Default-Werte*, *Pfadregeln*, *Listen-/Schleifenregeln*, *Filterregeln*, *Kardinalitätsregeln*) verwendet werden. Für jeden Regeltyp werden drei verschiedene Objekte gezeigt:

1. **Payload:** Der eingehende Datensatz
2. **Template-Submodel:** Ein Submodel-Template mit den relevanten Template-Qualifiers
3. **Instanz-Submodel:** Das daraus resultierende Instanz-Submodel

5.1 Ziel und Einordnung

Die Regeln dienen dazu, strukturierte Eingangsdaten deterministisch in [AAS](#)-konforme Submodels zu überführen. Anstatt Mapping-Konfigurationen in einer zentralen Datenbank zu speichern, werden die Regeln am Template als Qualifier abgelegt.

5.2 Regelmodell

Die Regeln werden immer in eigenen Template-Qualifiers hinterlegt. Jeder Qualifier enthält einen *type*, der den Regeltyp definiert, sowie einen *value*, der die Regelkonfiguration in Form einer Pfadangabe enthält.

Qualifier-Types: Für die Darstellung der Regeln werden folgende Template-Qualifier definiert:

- **SMT/MappingInfo** (Pfadregel): 1:1-Mapping von einem Datenpfad auf ein [SME](#).
- **SMT/CollectionMappingInfo** (Listenregel): Dupliziert ein [SME](#) für jedes Element einer Liste.
- **SMT/FilterMappingInfo** (Filterregel): Löscht ein [SME](#) abhängig von einer Bedingung.
- **SMT/Cardinality** (Optional/Mandatory): Definiert, ob in Verbindung mit einer anderen Mapping-Regel bei fehlendem Pfad ein [SME](#) übersprungen oder ein Fehler geworfen werden wird.

Pfadangaben: Pfade referenzieren Felder im Eingabe-JSON basierend auf der *JSONata*-Abfrage- und Transformationssprache [53]. Der Platzhalter `[*]` kennzeichnet Arrays und kann genutzt werden, um auf alle Elemente einer Liste gleichzeitig zuzugreifen. Als *ListIdentifier* wird der String bezeichnet, den alle Elemente und ihre Kinder in einer Liste gemeinsam haben. Konkret ist es der Pfad `CollectionMappingInfo`-Qualifiers *ohne* das letzte `[*]`. Er kann von den Kindern in den eigenen Qualifiers genutzt werden, um auf die jeweiligen Listenelemente zuzugreifen.

5.3 Notation der Submodels

Die Codebeispiele in diesem und dem folgenden Kapitel verwenden eine vereinfachte, kompakte Notation für AAS-Submodels, die sich an der im *AAS-Metamodell* [9] beschriebenen Struktur orientiert. Die Notation umfasst folgende Elemente:

- `[SM]`: Submodel (`<Kind>`)
- `[SMC]`: `SubmodelElementCollection`
- `[P]`: Property
- `[MQ: <Pfad>]`: Template-Qualifier `SMT/MappingInfo` mit Pfadangabe
- `[CMQ: <Pfad>]`: Template-Qualifier `SMT/CollectionMappingInfo` mit Pfadangabe
- `[FMQ: <Bedingung>]`: Template-Qualifier `SMT/FilterMappingInfo` mit Bedingung
- `[CQ: <Kardinalität>]`: Template-Qualifier `SMT/Cardinality` mit Kardinalität
- `= <Wert>`: Statischer Wert
- `(<Kind>)`: Art des Submodels (Template/Instance)

Für bessere Lesbarkeit wurden die Qualifier in die Zeile des jeweiligen `SME` integriert. Sie werden nur bei Template-Submodel gezeigt. Im Instanz-Submodel werden sie nicht dargestellt, da sie dort keine Bedeutung haben.

Zusätzlich sind die Submodels im Anhang als vollständige JSON-Objekte dargestellt.

5.4 Regeltypen mit Beispielen

Im Folgenden werden die vier Regeltypen jeweils kurz erläutert und mit Fall-Beispielen belegt.

5.4.1 Default-Werte (statisch)

Statische Werte werden direkt im Template hinterlegt. Es sind keine Eingabedaten erforderlich. Beim Erzeugen der Instanz wird der Wert unverändert übernommen.

Template-Submodel

```
1 [SM] ManufacturerInfo (Template)
2 +-- [P] Manufacturer = "XITASO GmbH"
```

5.1 Codeausschnitt: Default-Werte – Template-Submodel

Instanz-Submodel

```
1 [SM] ManufacturerInfo (Instance)
2 +-- [P] Manufacturer = "XITASO GmbH"
```

5.2 Codeausschnitt: Default-Werte – Instanz-Submodel

Die Codebeispiele 5.1 und 5.2 sind bis auf kind identisch, Qualifier sind nicht erforderlich.

5.4.2 Pfadregeln (dynamische Werte)

Dynamische Werte werden per SMT/MappingInfo an ein SME gebunden. Die Daten, die im Payload unter dem angegebenen Pfad gefunden werden, werden als Wert übernommen.

Payload

```
1 {
2   "car": {
3     "serialNo": "DE-00042"
4   }
5 }
```

5.3 Codeausschnitt: Pfadregel – Payload

Template-Submodel

```
1 [SM] Identification (Template)
2 +-- [P] serialNo [MQ: car.serialNo]
```

5.4 Codeausschnitt: Pfadregel – Template-Submodel

Instanz-Submodel

```
1 [SM] Identification (Instance)
2 +-- [P] serialNo = "DE-00042"
```

5.5 Codeausschnitt: Pfadregel – Instanz-Submodel

Die Codebeispiele 5.3-5.5 zeigen, wie der Wert DE-00042 aus der Payload per Pfadzuweisung in das Instanz-Submodel übernommen wird.

5.4.3 Listen-/Schleifenregeln (Duplizieren)

Listen werden über SMT/CollectionMappingInfo abgebildet. Der Qualifier sitzt am zu duplizierenden SME. Untergeordnete Elemente greifen per *ListIdentifier* in SMT/MappingInfo auf ihre Werte zu.

Payload

```
1 {
2   "car": {
3     "contacts": [
4       { "name": "Joe", "email": "joe@doe.com" },
5       { "name": "Jane", "email": "jane@doe.com" }
6     ]
7   }
8 }
```

5.6 Codeausschnitt: Listenregel – Payload

Template-Submodel

```

1 [SM] Contacts (Template)
2 +-- [SMC] contacts
3     +-- [SMC] contactPerson [CMQ: car.contacts[*]]
4         |-- [P] Name [MQ: car.contacts[*].name]
5         +-- [P] Email [MQ: car.contacts[*].email]

```

5.7 Codeausschnitt: Listenregel – Template-Submodel

Instanz-Submodel

```

1 [SM] Contacts (Instance)
2 +-- [SMC] contacts
3     |-- [SMC] contactPerson_0
4         |-- [P] Name = "Joe"
5         +-- [P] Email = "joe@doe.com"
6     +-- [SMC] contactPerson_1
7         |-- [P] Name = "Jane"
8         +-- [P] Email = "jane@doe.com"

```

5.8 Codeausschnitt: Listenregel – Instanz-Submodel

Die Codebeispiele 5.6-5.8 zeigen, wie für die zwei Elemente des contacts-Array in der Payload zwei SMCs im Instanz-Submodel angelegt werden und mit den jeweiligen Werten von Jane Doe und Joe Doe befüllt werden.

5.4.4 Filterregeln (bedingte Instanziierung)

Mit SMT/FilterMappingInfo werden SME nur dann erzeugt, wenn eine Bedingung erfüllt ist. Die in value angegebene Bedingung wird als *JSONata*-Ausdruck interpretiert. Ergibt die Auswertung true, wird das SME erzeugt, ergibt sie false nicht.

Payload

```

1 {
2   "car": {
3     "engineType": "electric",
4     "battery": { "capacityKWh": 90 },
5     "gasTank": { "capacityL": 50 }
6   }
7 }

```

5.9 Codeausschnitt: Filterregel – Payload

Template-Submodel

```

1 [SM] VehicleSpecs (Template)
2 |-- [SMC] BatteryInfo [FMQ: car.engineType = 'electric']
3 |   +-- [P] CapacityKWh [MQ: car.battery.capacityKWh]
4 +-- [SMC] GasTankInfo [FMQ: car.engineType = 'combustion']
5   +-- [P] CapacityL [MQ: car.gasTank.capacityL]

```

5.10 Codeausschnitt: Filterregel – Template-Submodel

Instanz-Submodel

```

1 [SM] VehicleSpecs (Instance)
2 +-- [SMC] BatteryInfo
3   +-- [P] CapacityKWh = 90

```

5.11 Codeausschnitt: Filterregel – Instanz-Submodel

Die Codebeispiele 5.9-5.11 zeigen, wie das Template-Submodel sowohl eine BatteryInfo- als auch eine GasTankInfo-SMC enthält. Die Qualifier SMT/FilterMappingInfo sorgen dafür, dass nur die SMC erzeugt wird, deren Bedingung im Payload erfüllt ist. Da im Beispiel der engineType auf electric gesetzt ist, wird nur die BatteryInfo in das Instanz-Submodel übernommen.

5.4.5 Kardinalitätsregeln (Optional/Verpflichtend)

Mit SMT/Cardinality wird definiert, ob ein [SME](#) verpflichtend (OneToOne) oder optional (ZeroToOne) ist. Bei verpflichtenden Feldern führt ein fehlender Pfad zum Abbruch der Submodel-Erzeugung. Bei optionalen Feldern wird das [SME](#) mit leerem Wert erstellt, falls der Pfad im Payload fehlt. Ist keine Kardinalität angegeben, wird das Feld als optional behandelt.

Payload

```
1 {  
2   "product": {  
3     "name": "FooBar Pro Max"  
4   }  
5 }
```

5.12 Codeausschnitt: Cardinality – Payload

Template-Submodel

```
1 [SM] ProductInfo (Template)  
2 |-- [P] ProductName [MQ: product.name] [CQ: One]  
3 +-- [P] ProductDescription [MQ: product.description] [CQ: ZeroToOne]
```

5.13 Codeausschnitt: Kardinalitätsregel – Template-Submodel

Instanz-Submodel

```
1 [SM] ProductInfo (Instance)
2 |-- [P] ProductName = "FooBar Pro Max"
3 +-- [P] ProductDescription = ""
```

5.14 Codeausschnitt: Kardinalitätsregel – Instanz-Submodel

Die Codebeispiele 5.12-5.14 zeigen das Verhalten bei fehlenden Pfaden: `ProductName` ist als verpflichtend (One) markiert und werden korrekt gemappt. `ProductDescription` ist optional (ZeroToOne) und wird trotz fehlendem Pfad im Payload in das Instanz-Submodel übernommen, jedoch mit leerem Wert. Wäre der verpflichtende Pfad `product.name` im Payload nicht vorhanden, würde die Submodel-Erzeugung mit einer strukturierten Fehlermeldung abbrechen.

5.5 Zusammenfassung

Die fünf Regeltypen decken die in Kapitel 4 spezifizierten Kernanforderungen ab: statische Vorgaben, 1:1-Pfadzuweisungen, die Iteration über Listen, bedingte Instanziierung, sowie die Kardinalitäten für optional/verpflichtende Felder. Die Ablage als Template-Qualifier ermöglicht eine transparente und testbare Erzeugung von Instanz-Submodels; die gezeigten Beispiele dienen als Referenz für Implementierung und Tests.

6 Prototypische Implementierung

Die in Kapitel 5 definierten Regeln bilden die Grundlage für die Implementierung der Rules-Engine. Dieses Kapitel beschreibt die Transformation der bestehenden Generator-Logik im *Eclipse-Mnestix*-Backend zu einer robusten, erweiterbaren Lösung, die AAS-v3-konforme Submodel-Instanzen erzeugt. Dabei soll ein lauffähiger Prototyp entwickelt werden, der die vorgeschlagene Architektur in einem Querschnitt demonstriert, aber nicht zwingend alle in Kapitel 4 erarbeiteten Anforderungen erfüllt.

Die Implementierung orientiert sich am *Pipeline-Pattern* [54], [55], um die bisher in einer einzigen Methode definierten Erzeugungslogik in klar abgegrenzte Komponenten zu unterteilen. Jeder Schritt der Pipeline übernimmt eine spezifische Verantwortlichkeit bei der Transformation von Submodel-Templates zu instanziierten Submodelsn.

Abschnitt 6.1 erläutert die Pipeline-Architektur der Rules-Engine und ihre Implementierung. Abschnitt 6.2 beschreibt wie sie in das bestehende *Eclipse-Mnestix*-Backend integriert wird und wie der Datenfluss bei der Generation eines Submodels abläuft. Zudem werden in Abschnitt 6.3 zwei wichtige Schritte der Pipeline im Detail erläutert. Zuletzt werden in Abschnitt 6.4 die vorgenommenen Änderungen an der Benutzeroberfläche beschrieben.

6.1 Pipeline-Architektur

Zur Lösung der identifizierten Probleme wird eine *Pipeline*- bzw. *Pipes-And-Filters*-Architektur nach Buschmann et al. [54] implementiert, welche erstmalig bei der Entwicklung des *Unix*-Betriebssystems eingesetzt wurde. Diese Architektur besteht aus einer Reihe von unabhängigen Verarbeitungsschritten (Filters), die nacheinander ausgeführt werden. Jeder Schritt übergibt seine Ausgabe über einen Kanal (Pipe) als Eingabe an den nächsten Schritt, wodurch ein klar definierter Datenfluss entsteht. Das Pipeline-Pattern ist besonders für Systeme geeignet, die Datenströme transformieren sollen [54].

Buschmann et al. [54] definieren sechs Schritte zur Implementierung einer Pipeline-Architektur:

1. **Aufteilung der Gesamtaufgabe:** Zerlegung des Gesamtprozesses in einzelne, unabhängige Schritte
2. **Definition der Formate:** Festlegung einheitlicher Schnittstellen für die Kommunikation zwischen den Schritten

3. **Implementierung der Kanäle:** Entwicklung der Mechanismen zur Datenübertragung zwischen den Schritten
4. **Implementierung der Filter:** Entwicklung der einzelnen Verarbeitungsschritte als eigenständige Komponenten
5. **Fehlerbehandlung und Logging:** Implementierung von Mechanismen zur Fehlererkennung und -behandlung auf Schritt-Ebene
6. **Zusammenstellen der Pipeline:** Orchestrierung der Schritte zu einer vollständigen Pipeline

Im Folgenden werden anhand dieser Schritte die wesentlichen Aspekte der Pipeline-Architektur für die Rules-Engine erläutert. Die konkrete Implementierung orientiert sich an einem Blog-Post von *Karabulut* [55], welcher die Umsetzung des Pipeline-Patterns in *.NET* beschreibt.

6.1.1 Schritt 1: Aufteilung der Gesamtaufgabe

Die Transformation eines Template-Submodels zu einer Instanz wird in folgende unabhängige Verarbeitungsschritte zerlegt:

1. **DeepCloneTemplateAasGeneratorPipelineStep:** Kopie des Submodel-Templates als Arbeitsgrundlage
2. **SetKindInstanceAasGeneratorPipelineStep:** Setzen von kind zu „Instance“
3. **DuplicateCollectionsAasGeneratorPipelineStep:** Duplizierung von Elementen basierend auf SMT/CollectionMappingInfo-Qualifiers
4. **MapDataToInstanceAasGeneratorPipelineStep:** Mapping von Daten basierend auf SMT/MappingInfo-Qualifiers
5. **RemoveTopLevelQualifiersAasGeneratorPipelineStep:** Entfernung der Submodel-Qualifier
6. **ReplaceIdentificationAasGeneratorPipelineStep:** Zuweisung der neuen Submodel-ID

6.1.2 Schritt 2: Definition der Formate

Obwohl nicht jeder Schritt das selbe Format für Ein- und Ausgabedaten haben muss, bietet ein einheitliches Format mehr Flexibilität [54]. Deshalb wird für die Kommunikation zwischen den Schritten ein einheitliches Kontextobjekt definiert, welches in 6.1 dargestellt ist.

```

1 public class AasGenerationContext
2 {
3     // Immutable inputs
4     public JObject Template { get; }
5     public JObject Data { get; }
6     public string Language { get; }
7     public string NewSubmodelId { get; }
8
9     // Logger for diagnostics
10    private readonly ILogger<SubmodelMappingContext> _logger;
11
12    // Mutable working object
13    public JObject SubmodelInstance { get; set; }
14
15    // Save logs for each step
16    public IList<string> Logs { get; } = new List<string>();
17    public void Log(string message)
18    {
19        Logs.Add($"[{DateTime.UtcNow}] - {message}");
20        _logger.LogInformation(message);
21    }
22
23    // The currently processed qualifiers (for error reporting)
24    public JToken Qualifier { get; set; } = new JObject();
25 }

```

6.1 Codeausschnitt: Pipeline-Kontext

Das Kontext-Objekt transportiert den aktuellen Stand des Submodels in `SubmodelInstance`. Daten, die sich während der gesamten Pipeline nicht ändern sollen, wie das Submodel-Template, die Eingabedaten, die Sprache und die neue Submodel-ID, werden als unveränderliche Eigenschaften definiert. Zudem enthält das Kontext-Objekt die Möglichkeit Informationen zu loggen, die dann an den Standard-Logkanal weitergegeben werden und in einer internen Liste gespeichert werden. Indem der aktuell zu behandelnde Qualifier auch im Kontext gespeichert wird, kann er bei einem Fehler auch über die [API](#) zurückgegeben werden (vgl. [6.1.5](#)).

6.1.3 Schritt 3: Implementierung der Kanäle

Die Datenübertragung zwischen Schritten erfolgt über ein zentrales Pipeline-Objekt:


```

1 internal sealed class Pipeline<TContext> : IPipeline<TContext>
2 {
3     private readonly IReadOnlyList<IPipelineStep<TContext>> _steps;
4
5     public Pipeline(IReadOnlyList<IPipelineStep<TContext>> steps)
6     {
7         _steps = steps;
8     }
9
10    public async Task<TContext> RunAsync(TContext context)
11    {
12        var current = context;
13        foreach (var step in _steps)
14        {
15            current = await
16                step.ExecuteAsync(current).ConfigureAwait(false);
17        }
18        return current;
19    }
20 }

```

6.2 Codeausschnitt: Pipeline

Alle Schritte sind in einer privaten Liste registriert und werden nacheinander ausgeführt. Jeder Schritt erhält das vollständige Kontext-Objekt, kann es modifizieren und gibt es an den nächsten Schritt weiter. Es werden keine Schritte parallel ausgeführt, aber sie können von der *.NET*-Umgebung in neue Threads ausgelagert werden, wenn dies für Performance-Zwecke notwendig ist [56].

6.1.4 Schritt 4: Implementierung der Filter

Die einzelnen Verarbeitungsschritte werden von dem *IPipelineStep*-Interface abgeleitet und als eigenständige Klassen implementiert.

```

1 public interface IPipelineStep<TContext>
2 {
3     Task<TContext> ExecuteAsync(TContext context);
4 }

```

6.3 Codeausschnitt: IPipelineStep-Interface

Wie in Codeausschnitt 6.3 gezeigt, implementiert jeder Schritt eine *ExecuteAsync*-Methode, die einen *Task* zurückgibt. Wie im vorherigen Schritt beschrieben, erlaubt die Verwendung von

.Net-Tasks dem Framework flexibleres Multithreading zu betreiben. Perspektivisch könnten mit minimalen Änderungen die Schritte auch parallelisiert werden. In Abschnitt 6.3 werden zwei Schritte exemplarisch im Detail beschrieben.

6.1.5 Schritt 5: Fehlerbehandlung und Logging

Die Pipeline implementiert strukturierte Fehlerbehandlung auf Schritt-Ebene mit spezifischen Exceptions und der Möglichkeit, für Debugging relevante Informationen im Kontext zu sammeln.

```

1 public class SubmodelDataToInstanceMapperException : Exception
2 {
3     public SubmodelMappingContext? Context { get; set; }
4     // ...
5     public SubmodelDataToInstanceMapperException(string? message,
6         Exception? innerException, SubmodelMappingContext? ctx) :
7         base(message, innerException)
8     {
9         Context = ctx;
10    }
11    public SubmodelDataToInstanceMapperException(string? message,
12        SubmodelMappingContext? ctx) : base(message)
13    {
14        Context = ctx;
15    }
16    // ...
17 }

```

6.4 Codeausschnitt: Strukturierte Fehlerbehandlung

Die Schritte übergeben den geworfenen Fehlermeldungen den aktuellen Kontext und den fehlerhaften Qualifier, um die Diagnose zu erleichtern. Die Pipeline wird immer abgebrochen, wenn ein Fehler auftritt. Anschließend wird der Fehler über die aufrufende Methode bis zur [API](#) weitergereicht. Es wird davon ausgegangen, dass Fehler in der Regel auf fehlerhafte Eingabedaten zurückzuführen sind und daher nicht durch die Pipeline selbst behoben werden können.

6.1.6 Schritt 6: Zusammenstellen der Pipeline

Die vollständige Pipeline wird von einem PipelineBuilder-Konstruktor zusammengebaut:

```

1 internal sealed class PipelineBuilder<TContext>
2 {
3     private readonly List<Type> _stepTypes = new();
4
5     public PipelineBuilder<TContext> Use<TStep>() where TStep : class
6         , IPipelineStep<TContext>, new()
7     {
8         _stepTypes.Add(typeof(TStep));
9         return this;
10    }
11
12    public IPipeline<TContext> Build()
13    {
14        var steps = new List<IPipelineStep<TContext>>();
15        foreach (var stepType in _stepTypes)
16        {
17            var step = (IPipelineStep<TContext>)Activator.
18                CreateInstance(stepType!);
19            steps.Add(step);
20        }
21        return new Pipeline<TContext>(steps);
22    }
23 }

```

6.5 Codeausschnitt: Pipeline-Builder

Dem Builder können mit der `Use<TStep>()`-Methode beliebig viele Schritte hinzugefügt werden. Mit der `Build`-Methode kann die Pipeline erstellt werden. Die Schritte können wie folgt in der gewünschten Reihenfolge registriert werden.

```

1 public sealed class SubmodelDataToInstanceMapper :
   ISubmodelDataToInstanceMapper
2 {
3     // ...
4     public JObject CreateSubmodelInstanceFromDataJson(JObject
       submodelTemplate, JObject data, string language, string
       newSubmodelId)
5     {
6         // ...
7         var context = new SubmodelMappingContext(submodelTemplate,
           data, language, newSubmodelId, logger);
8
9         // Build pipeline with all the steps in the correct order
10        var pipeline = new MnestixCore.AasGenerator.Pipelines.Core.
           PipelineBuilder<SubmodelMappingContext>()
11            .Use<DeepCloneTemplateAasGeneratorPipelineStep>()
12            .Use<SetKindInstanceAasGeneratorPipelineStep>()
13            .Use<DuplicateCollectionsAasGeneratorPipelineStep>()
14            .Use<MapDataToInstanceAasGeneratorPipelineStep>()
15            .Use<RemoveTopLevelQualifiersAasGeneratorPipelineStep>()
16            .Use<ReplaceIdentificationAasGeneratorPipelineStep>()
17            .Build();
18
19        var resultCtx = pipeline.RunAsync(context).GetAwaiter().
           GetResult();
20        return resultCtx.SubmodelInstance;
21    }
22 }

```

6.6 Codeausschnitt: SubModelDataToInstanceMapper

6.1.7 Fazit

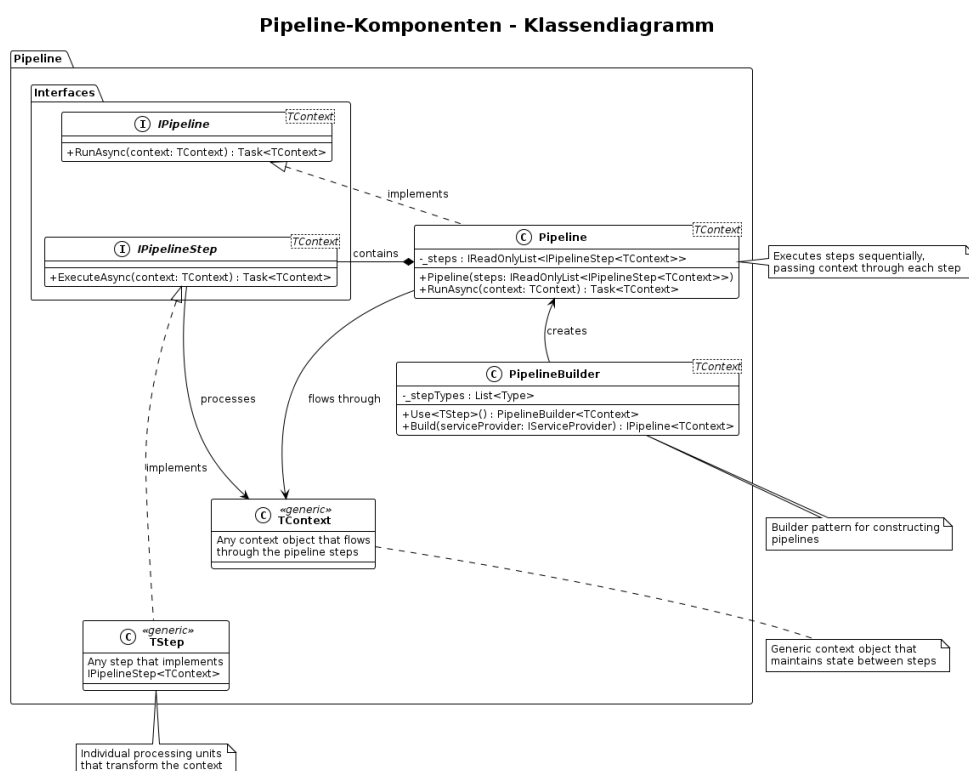
Die neu entwickelte Architektur soll die bisherigen Probleme der Generator-Logik lösen, indem sie eine klare Trennung der Verantwortlichkeiten, eine einfache Erweiterbarkeit und eine robuste Fehlerbehandlung ermöglicht. Bei der Umsetzung soll darauf geachtet werden, möglichst viel des aktuell vorhandenen Quellcodes wiederzuverwenden.

6.2 Integration in das Eclipse-Mnestix-Backend

Im Folgenden wird zunächst beschrieben wie die Pipeline-Architektur in das bestehende *Eclipse-Mnestix*-Backend integriert wird (Statische Integration). Anschließend wird der Datenfluss bei der Generierung eines Submodels erläutert (Dynamische Integration).

6.2.1 Statische Integration

Für die Implementierung der Pipeline-Architektur wurden generische Komponenten erstellt, die in verschiedenen Bereichen wiederverwendet werden können und deshalb im Ordner `MnestixCore/Shared/Pipeline` abgelegt wurden. Bei den Komponenten handelt es sich um Interfaces für eine Pipeline und einen Schritt, sowie dem `PipelineBuilder`, der Schritte registriert und dann ein Objekt der Klasse `Pipeline` erstellt. Die Struktur dieser Komponenten ist in Abbildung 6.1 dargestellt.



6.1 Abbildung: Klassendiagramm der Pipeline-Komponenten

Die Komponenten die direkt für die Rules-Engine entwickelt wurden, befinden sich im Ordner `MnestixCore/AasGenerator/`. Die Struktur dieser Komponenten ist wie folgt organisiert:

```

1 AasGenerator/
2 |-- AasGenerator.cs
3 |-- AasGeneratorResult.cs
4 |-- Interfaces/
5 |   |-- IAasGenerator.cs
6 |   +-- ISubmodelDataToInstanceMapper.cs
7 +-- SubmodelDataToInstanceMapper/
8     |-- Steps/
9     |   |-- DeepCloneTemplateStep.cs
10    |   |-- DuplicateCollectionsStep.cs
11    |   |-- MapDataToInstanceStep.cs
12    |   |-- RemoveTopLevelQualifiersStep.cs
13    |   |-- ReplaceIdentificationStep.cs
14    |   +-- SetKindInstanceStep.cs
15    |-- SubmodelDataToInstanceMapper.cs
16    +-- SubmodelMappingContext.cs

```

6.7 Codeausschnitt: Dateistruktur des AAS Generator

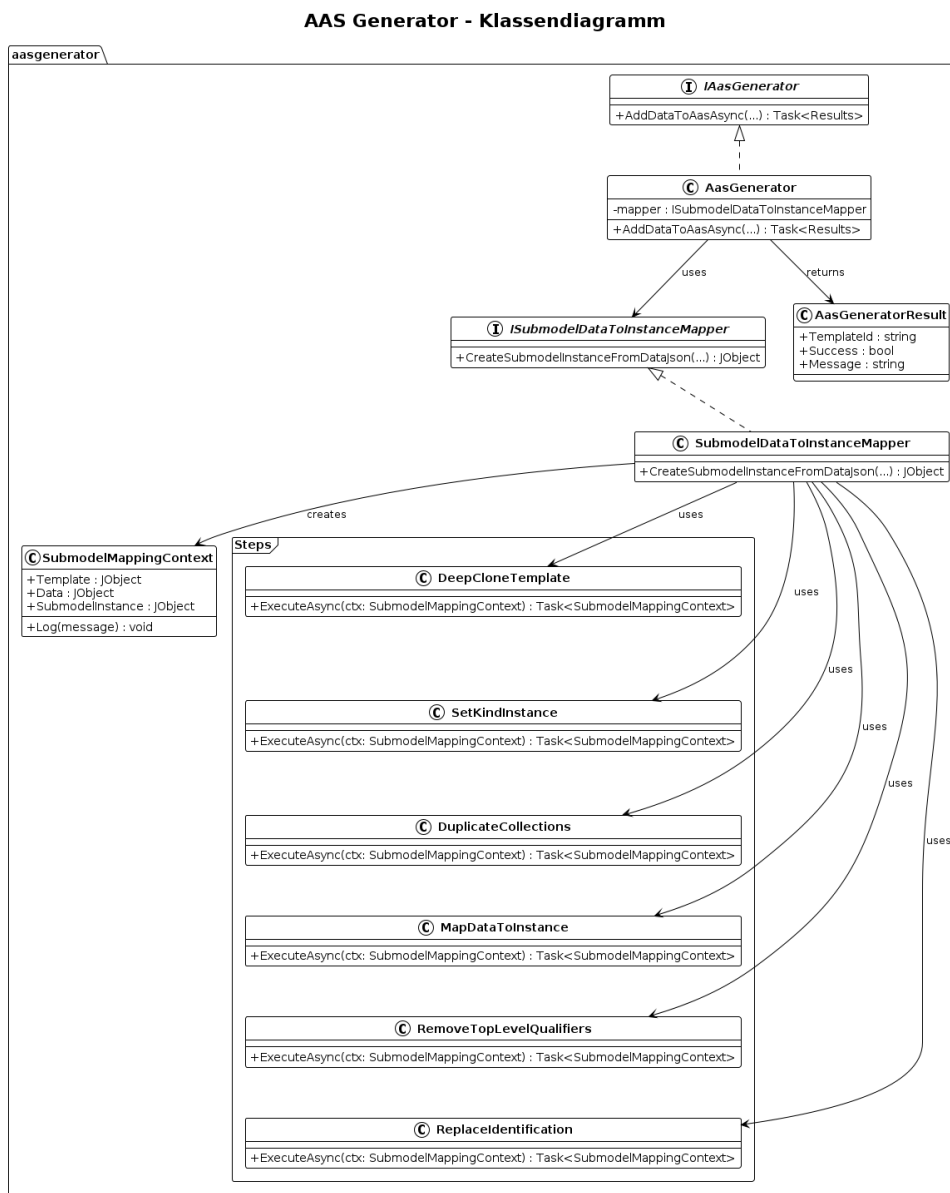
In Abbildung 6.2 ist ein Klassendiagramm der AAS-Generator-Komponenten dargestellt. Die Klasse `AasGenerator` stellt die Hauptschnittstelle für die Generierung von [AASs](#) dar. Sie verwendet intern den `SubmodelDataToInstanceMapper`, der die Pipeline zur Transformation von Submodel-Templates in Instanzen anstößt. Jeder Verarbeitungsschritt der Pipeline ist als eigene Komponente implementiert.

6.2.2 Dynamische Integration

Die dynamische Integration beschreibt den Ablauf der Submodel-Generierung im *Eclipse-Mnestix*-Backend vom Empfang der Anfrage in der [API](#) bis zur Rückgabe des generierten Submodels.

Dieser in Abbildung 6.3 dargestellte Ablauf wird im Folgenden erläutert:

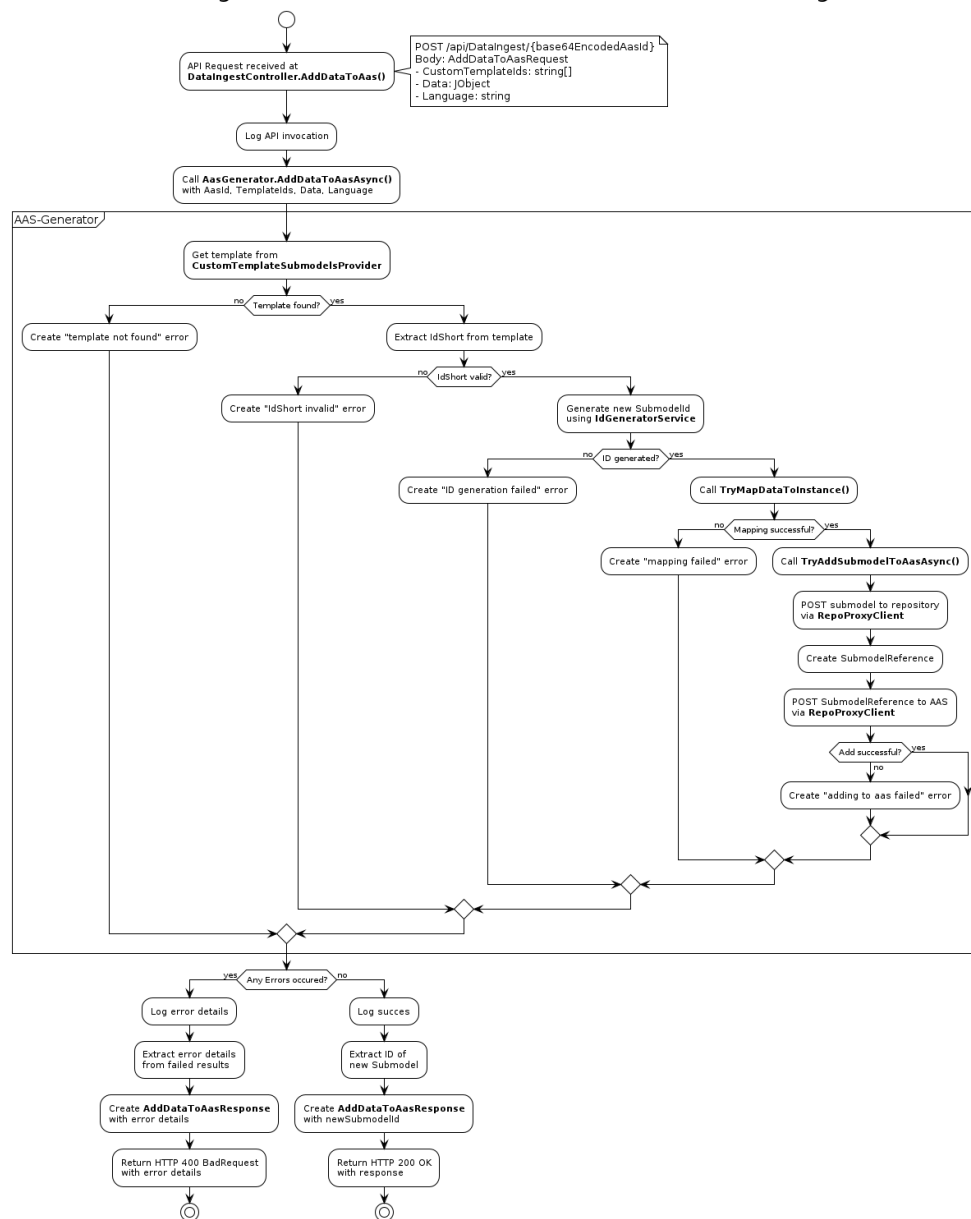
Zunächst empfängt die [API](#) eine Anfrage zur Generierung eines Submodels. Diese Anfrage enthält das Submodel-Template, die Eingabedaten, die verwendete Sprache und die neue Submodel-ID. Die [API](#) validiert die Anfrage und leitet sie an den `AasGenerator` weiter. Der `AasGenerator` fragt das Template an, überprüft ob es eine `idShort` besitzt und generiert eine [ID](#) für das zu erstellende Submodel. Anschließend wird ein neues Kontext-Objekt mit den übergebenen Daten erstellt und die Pipeline mit den definierten Verarbeitungsschritten initialisiert. Danach wird die Pipeline ausgeführt, wobei jeder Schritt das Kontext-Objekt modifiziert und an den nächsten Schritt weitergibt. Nach Abschluss der Pipeline enthält das Kontext-Objekt das generierte Submodel, das an den `AasGenerator` zurückgegeben wird. Das Submodel wird der gewünschten [AAS](#) hinzugefügt. Zuletzt wird die [ID](#) des generierten Submodels über die [API](#) an den Client gesendet. Tritt während der Pipeline-Ausführung ein Fehler auf, wird dieser abgefangen und über die [API](#)



6.2 Abbildung: Klassendiagramm der AAS-Generator-Komponenten

als Fehlerantwort samt den detaillierten Fehlermeldungen, die im Kontext-Objekt gesammelt wurden, an den Client zurückgeben.

Generierung eines Submodels mit dem AAS Generator - Aktivitätsdiagramm



6.3 Abbildung: Ablauf der Submodel-Generierung im Eclipse-Mnestix-Backend

6.3 Implementierung ausgewählter Pipeline-Schritte

Der Fokus liegt auf den Schritten `DuplicateCollectionsAasGeneratorPipelineStep` und `MapDataToInstanceAasGeneratorPipelineStep`, da diese die Kernfunktionalität der Rules-Engine abbilden. Die Schritte sollen anhand des folgenden Beispiels erläutert werden:

Das Template in Listing 6.8 beschreibt ein Submodel mit einer Liste von Kontakten, wobei jeder Kontakt einen Namen und eine Liste von Telefonnummern enthält. Jede Telefonnummer besteht

aus einem Namen und einem Wert. Die SMT/CollectionMappingInfo-Qualifiers geben an, dass die `contact`- und `phoneNumber`-Elemente zu duplizierende Listenelemente sind, die auf die entsprechenden Arrays in der Eingabe-Payload abgebildet werden sollen. Die SMT/MappingInfo-Qualifiers geben die Pfade zu den Datenfeldern in der Eingabe-Payload an, die den jeweiligen Eigenschaften zugeordnet werden sollen.

Die Eingabe-Payload in Listing 6.9 enthält zwei Kontakte: *SpongeBob* mit zwei Telefonnummern und *Squidward* mit drei Telefonnummern.

```

1 [SM] Information
2 +-- [SMC] contacts
3     +-- [SMC] contact [CMQ: data.contacts[*]]
4         |-- [P] Name [MQ: data.contacts[*].name]
5     +-- [SMC] phoneNumbers
6         +-- [SMC] phoneNumber [CMQ: data.contacts[*].
           phone_numbers[*]]
7             |-- [P] Name [MQ: data.contacts[*].phone_numbers[*].
               name]
8             +-- [P] Value [MQ: data.contacts[*].phone_numbers[*]
               .value]

```

6.8 Codeausschnitt: Submodel-Template

Eingabe-Payload:

```
1  "data": {
2    "contacts": [
3      {
4        "name": "SpongeBob",
5        "phone_numbers": [
6          {
7            "name": "Office",
8            "value": "+1 234 5678"
9          },
10         {
11           "name": "Cell",
12           "value": "+1 190 7161"
13         }
14       ]
15     },
16     {
17       "name": "Squidward",
18       "phone_numbers": [
19         {
20           "name": "Office",
21           "value": "+2 234 5678"
22         },
23         {
24           "name": "Home",
25           "value": "+1 893 2168"
26         },
27         {
28           "name": "Cell",
29           "value": "+2 190 7161"
30         }
31       ]
32     }
33   ]
34 }
```

6.9 Codeausschnitt: JSON-Eingabe-Payload

6.3.1 DuplicateCollectionsAasGeneratorPipelineStep

Dieser Schritt ist für zwei Aufgaben verantwortlich: Duplizieren der Listen-Elemente basierend auf den SMT/CollectionMappingInfo-Qualifiers und Anpassen der generischen Pfade in den SMT/MappingInfo-Qualifiern der Kinder-Elemente.

Aus den zuvor definiertem Submodel-Template 6.8 und Eingabe-Payload 6.9 soll das folgendes Submodel generiert werden:

```

1  [SM] Information
2  +-- [SMC] contacts
3      |-- [SMC] contact_0
4          |-- [P] Name [MQ: data.contacts[0].name]
5          +-- [SMC] phoneNumbers
6              |-- [SMC] phoneNumber_0
7                  |-- [P] Name [MQ: data.contacts[0].phone_numbers[0].
                        name]
8                  +-- [P] Value [MQ: data.contacts[0].phone_numbers[0]
                        .value]
9              +-- [SMC] phoneNumber_1
10                 |-- [P] Name [MQ: data.contacts[0].phone_numbers[1].
                        name]
11                 +-- [P] Value [MQ: data.contacts[0].phone_numbers[1]
                        .value]
12  +-- [SMC] contact_1
13      |-- [P] Name [MQ: data.contacts[1].name]
14      +-- [SMC] phoneNumbers
15          |-- [SMC] phoneNumber_0
16              |-- [P] Name [MQ: data.contacts[1].phone_numbers[0].
                        name]
17              +-- [P] Value [MQ: data.contacts[1].phone_numbers[0]
                        .value]
18          |-- [SMC] phoneNumber_1
19              |-- [P] Name [MQ: data.contacts[1].phone_numbers[1].
                        name]
20              +-- [P] Value [MQ: data.contacts[1].phone_numbers[1]
                        .value]
21          +-- [SMC] phoneNumber_2
22              |-- [P] Name [MQ: data.contacts[1].phone_numbers[2].
                        name]
23              +-- [P] Value [MQ: data.contacts[1].phone_numbers[2]
                        .value]

```

6.10 Codeausschnitt: Gewünschtes Submodel

Der Codeausschnitt 6.10 zeigt, dass für jedes Element in der Payload eine Property angelegt werden soll, die mit statischen SMT/MappingInfo-Qualifiern ausgestattet sind.

Für den Algorithmus wurde ein rekursiver Ansatz gewählt, da die Kopie eines Elements dazu führen kann, dass sich weitere Elemente im duplizierten Element befinden, die ebenfalls dupliziert werden müssen. Indem nach jedem Schritt der Algorithmus rekursiv aufgerufen wird und erneut nach SMT/CollectionMappingInfo-Qualifiers gesucht wird, wird sichergestellt, dass

alle Ebenen der Verschachtelung korrekt verarbeitet werden.

Der Algorithmus läuft wie folgt ab:

1. Suche nach SMT/CollectionMappingInfo-Qualifiers. Wenn keine gefunden, springe zu Schritt 8.
2. Wähle den Qualifier aus, der am flachsten in der Baumstruktur liegt.
3. Finde die zugehörige Liste im Payload. Wenn nicht gefunden, siehe Anmerkung unten.
4. Bestimme die Länge dieser Liste - die sog. `collectionLength`.
5. Wiederhole `collectionLength` mal:
 - a) Dupliziere das `SME`, an dem der SMT/CollectionMappingInfo-Qualifier hängt.
 - b) Passe die `idShort` an.
 - c) Passe die generischen Pfade in allen Qualifiers im duplizierten Objekt an.
 - d) Füge das duplizierte Objekt im Submodel ein.
6. Markiere den aktuellen Qualifier als bereits bearbeitet.
7. Rufe die Funktion rekursiv auf. Springe zu Schritt 1.
8. Entferne alle Markierungen der bearbeiteten Qualifiers.
9. Gib das bearbeitete Submodel zurück.

Falls in Schritt 3 die Liste im Payload nicht gefunden wird und das Element verpflichtend ist, wird eine `SubmodelDataToInstanceMapperException` geworfen. Ist das Element optional, wird keine Duplizierung durchgeführt.

Im Anhang ist der vollständige Quellcode, sowie ein ausführliches Ablaufdiagramm des Schrittes abgelegt.

6.3.2 MapDataToInstanceAasGeneratorPipelineStep

Dieser Schritt extrahiert die Daten aus der Eingabe-Payload und füllt die Werte in die entsprechenden Elemente des Submodels ein. Dabei werden alle Elemente mit einem SMT/MappingInfo-Qualifier berücksichtigt.

In diesem Schritt werden alle SMT/MappingInfo-Qualifiers im Submodel gesucht, die entsprechenden Werte aus der Payload extrahiert und in die `value`-Felder der Elemente eingesetzt. Nach diesem Schritt enthalten alle Properties die tatsächlichen Daten anstelle von leeren Strings.

Aus dem im vorherigen Schritt erstellten Submodel (vgl. Codeausschnitt 6.10) und der Eingabe-Payload (vgl. Codeausschnitt 6.9) soll das folgende Submodel generiert werden:

```

1  [SM] NestedListTemplate
2  +-- [SMC] contacts
3      |-- [SMC] contact_0
4          |-- [P] Name = "SpongeBob"
5          +-- [SMC] phoneNumbers
6              |-- [SMC] phoneNumber_0
7                  |-- [P] Name = "Office"
8                  +-- [P] Value = "+1 234 5678"
9              +-- [SMC] phoneNumber_1
10                 |-- [P] Name = "Cell"
11                 +-- [P] Value = "+1 190 7161"
12  +-- [SMC] contact_1
13      |-- [P] Name = "Squidward"
14      +-- [SMC] phoneNumbers
15          |-- [SMC] phoneNumber_0
16              |-- [P] Name = "Office"
17              +-- [P] Value = "+2 234 5678"
18          |-- [SMC] phoneNumber_1
19              |-- [P] Name = "Home"
20              +-- [P] Value = "+1 893 2168"
21          +-- [SMC] phoneNumber_2
22              |-- [P] Name = "Cell"
23              +-- [P] Value = "+2 190 7161"

```

6.11 Codeausschnitt: Gewünschtes Submodel

Der Algorithmus für das Mapping läuft iterativ über alle gefundenen SMT/MappingInfo-Qualifiers ab:

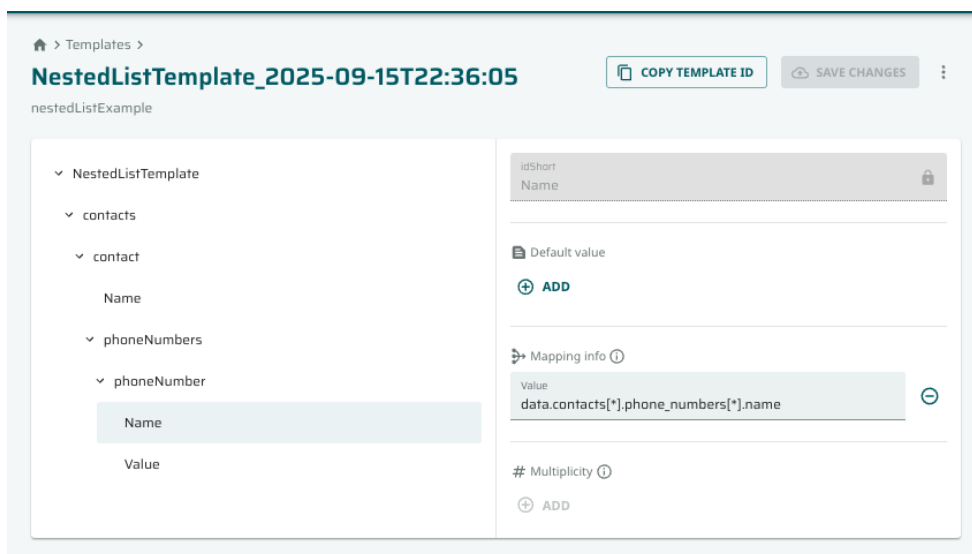
1. Suche alle SMT/MappingInfo-Qualifiers im Submodel.
2. Für jeden gefundenen Qualifier:
 - a) Extrahiere den Mapping-Pfad aus dem Qualifier.
 - b) Extrahiere den Wert aus der Payload unter Verwendung des Mapping-Pfads. Wenn nicht gefunden, siehe Anmerkung unten
 - c) Setze den extrahierten Wert in das value-Feld des Elements ein.
3. Gib das Submodel mit befüllten Werten zurück.

Falls in Schritt 2b der Wert in der Payload nicht gefunden wird und das Element verpflichtend ist, wird eine `SubmodelDataToInstanceMapperException` geworfen. Ist das Element optional, wird kein Wert gesetzt.

Der vollständige Quellcode dieses Schrittes befindet sich im Anhang.

6.4 Implementierung der GUI

Der *AAS Browser*, der für *Eclipse Mnestix* eine Weboberfläche bietet, unterstützt die Darstellung von *AASs* und deren Submodelsn. Submodel-Templates können in einem extra Abschnitt verwaltet werden. Bei der Erstellung der Templates in der *GUI* können verschiedene *IDTA*-Submodel-Templates verwendet werden. Die Ansicht auf ein einzelnes Submodel-Template ist in Abbildung 6.4 dargestellt [50].

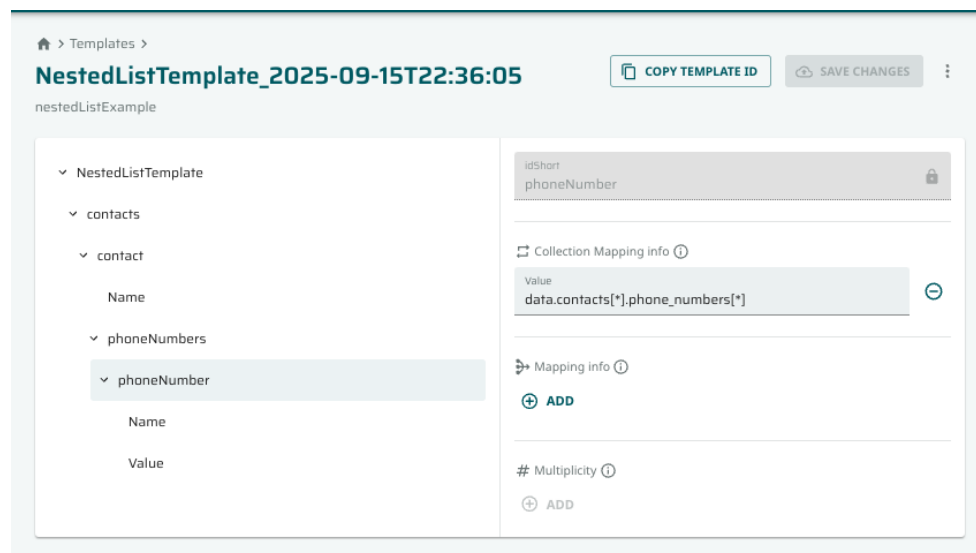


6.4 Abbildung: Ansicht einer Property eines Submodel-Templates im AAS Browser [50]

Es wird die Struktur des Submodel-Templates in einer Baumansicht dargestellt, wobei sich einzelne *SMEs* detailliert anzeigen lassen. In dieser Detailansicht können default Werte (bei Properties) und Mapping-Qualifiers bearbeitet werden, indem das entsprechende Feld befüllt wird. Zudem wird die idShort und Kardinalität (hier *Multiplicity*) des Elements angezeigt.

Da der Fokus dieser Arbeit auf der Implementierung der *REST-API* basierten Rules-Engine liegt, sollen nur minimale Änderungen an der Benutzeroberfläche vorgenommen werden. Konkret soll es die Möglichkeit geben *SMT/CollectionMappingInfo*-Qualifiers für die Elemente zu verwalten.

Dafür wurde eine neue Komponente `CollectionMappingInfoEditComponent.tsx` erstellt, die im Grunde zu der bestehenden `MappingInfoEditComponent.tsx` identisch ist. Der einzige Unterschied besteht darin, dass der Titel, die Beschreibung und der verwendete Qualifertype im Hintergrund angepasst wurden. Die Abbildung 6.5 zeigt die neue Komponente in der Detailansicht einer *SMC*.



6.5 Abbildung: Neue Ansicht einer [SMC](#) eines Submodel-Templates im AAS Browser [50]

7 Evaluation

In diesem Kapitel wird das in Kapitel 5 definierte Regelwerk und die in Kapitel 6 beschriebene Implementierung anhand der in Kapitel 4 definierten Akzeptanzkriterien evaluiert. Zunächst werden in Abschnitt 7.1 für die Akzeptanzkriterien, die mit einem Test überprüft werden können, die Testfälle beschrieben und die Ergebnisse dargestellt. Anschließend wird in Abschnitt 7.2 auf die Akzeptanzkriterien eingegangen, die nicht mit einem Test überprüft werden können. Zuletzt werden in Abschnitt 7.3 die Ergebnisse diskutiert.

7.1 Verifikation durch Tests

Für den Großteil der Akzeptanzkriterien wurde die Überprüfungsmethode **Test** gewählt. Dabei kann es sich um automatisierte Tests oder manuelle Tests handeln. In Abschnitt 7.1.1 werden die automatisierten Tests beschrieben, die in der Implementierung enthalten sind. Abschnitt 7.1.2 beschreibt die manuell durchgeführten Tests.

7.1.1 Automatisierte Tests

Für die Rules-Engine wurden automatisierte Tests implementiert, die die in Kapitel 4 definierten Akzeptanzkriterien verifizieren. Die Tests befinden sich im Verzeichnis `Core.Tests/AasGenerator` und können im Anhang gefunden werden. Die folgenden Akzeptanzkriterien konnten durch diese Tests überprüft werden:

AC-01 Pfadmapping korrekt

Test: `MandatoryAndOptionalField_Success`

Ergebnis: Bestanden

AC-02 Listen-Mapping korrekt

Test: `InputNestedList_Success`

Ergebnis: Bestanden

AC-03 Filter-Mapping korrekt

Test: `InputFilter_Success`

Ergebnis: Fehlgeschlagen, da die Filterregel aktuell nicht implementiert ist.

AC-04 Optionales Feld fehlt beim Pfadmapping

Test: InputOnlyMandatoryField_Success

Ergebnis: Bestanden

AC-05 Pflichtfeld fehlt beim Pfadmapping

Test: InputOnlyOptionalField_ShouldFail

Ergebnis: Bestanden

AC-07 Typ→Instanz-Erzeugung

Test: Wird in allen vorherigen Tests mit abgeprüft

Ergebnis: Bestanden

Für **AC-08** (Template-CRUD) wurden keine neuen Tests implementiert, da die Funktionalität nicht den **AAS** Generator sondern den *Template Builder* betrifft. In `Core.Tests/TemplateBuilder` befinden sich jedoch bereits Tests für die Template-Verwaltung. Diese Tests decken nur das Erstellen und Lesen von Templates ab, was durch die Analyse des Quellcodes bestätigt wird. Dieses Kriterium ist somit nur teilweise erfüllt.

7.1.2 Manuelle Tests

Für die folgenden Akzeptanzkriterien wurden manuelle Tests durchgeführt:

AC-09 Data-Ingest Roundtrip & AC-10 Strukturierte Fehlermeldung & Logging

Für diese **ACs** wurden **API**-Aufrufe mit dem Tool *Insomnia* durchgeführt. Die *Insomnia*-Projektdatei mit den definierten Requests kann im Anhang gefunden werden.

Für den Test von **AC-09** wurde ein Submodel-Template mit den in Kapitel 4 beschriebenen Regeln erstellt. Die **ID** dieses Templates, passende Eingabedaten und die **ID** des Ziel-**AAS** wurden der Anfrage mitgegeben. Die Antwort hat den Statuscode 200 und enthält die **ID** des erstellten Submodels. Weitere Analyse zeigt, dass das Submodel korrekt erstellt und der Ziel-**AAS** hinzugefügt wurde.

Für den Test von **AC-10** wurde ein Submodel-Template mit einer Pfadregel auf einen im Eingabe-**JSON** nicht vorhandenen Datenpfad erstellt. Die **ID** dieses Templates, passende Eingabedaten und die **ID** des Ziel-**AAS** wurden der Anfrage mitgegeben. Die Antwort hat den Statuscode 400 und enthält den nicht gefundenen Pfad sowie den betreffenden Qualifier. In den Logs befindet sich ein entsprechender Logeintrag mit denselben Informationen.

AC-12 Performanceziel

Für die Performance wurde ein Test hinzugefügt, der die Zeit misst, die benötigt wird, um ein Submodel mit einer [SMC](#) mit 10.000 Elementen zu generieren. Als Referenzhardware wurde ein Apple MacBookPro mit M4 Pro Prozessor und 24GB RAM verwendet. Das Ergebnis lag bei etwa 2.8 Sekunden, was das Performanceziel von unter 10 Sekunden deutlich erfüllt.

AC-13 Testbarkeit/Wartbarkeit

Für die Testbarkeit und Wartbarkeit wurde die Testabdeckung (Code-Coverage) betrachtet. Die Testabdeckung gibt an, wie viel Prozent des Quellcodes durch Tests abgedeckt werden. Es wird dabei zwischen der *Line-Coverage*, die angibt wie viel Prozent der Quellcodezeilen durch Tests ausgeführt werden und der *Branch-Coverage*, die angibt wie viel Prozent der Verzweigungen (z.B. if-Anweisungen) durch Tests abgedeckt werden, unterschieden [57].

Die Dateien im Verzeichnis `MnestixCore/AasGenerator` haben eine Line-Coverage von 80.0% und eine Branch-Coverage von 54.6% im Vergleich zu einer Line-Coverage von 22.8% und einer Branch-Coverage von 22.0% im gesamten Projekt. Die niedrige Branch-Coverage lässt jedoch darauf schließen, dass nicht alle Edge-Cases abgedeckt sind. Insgesamt kann das Kriterium somit als teilweise erfüllt angesehen werden.

7.2 Verifikation durch Analyse und Demonstration

Nicht alle Akzeptanzkriterien können durch Tests überprüft werden. Einige Kriterien erfordern eine Analyse des Quellcodes oder eine Demonstration der Funktionalität. Die folgenden Akzeptanzkriterien wurden auf diese Weise überprüft:

AC-06 Konformität AAS-Metamodell

Die Konformität des erzeugten Submodels mit dem [AAS](#)-Metamodell v3.* [9] wurde durch die Validierung des erzeugten Submodels mit dem offiziellen [AAS-v3-JSON-Schema](#) der [IDTA](#) [58] im Online-Tool [JSON Schema Validator](#) überprüft. Das erzeugte Submodel ist schema-valide und entspricht somit dem Metamodell. Allerdings sind die Template-Qualifier noch vorhanden, was nach der Instanziierung nicht der Fall sein sollte. Dieses Kriterium ist somit nur teilweise erfüllt.

AC-11 UI-Übersichten

Abschnitt 6.4 beschreibt die in der Implementierung enthaltenen UI-Übersichten. Diese wurden durch eine Demonstration der Funktionalität überprüft. Das Kriterium ist somit erfüllt.

AC-14 Erweiterbarkeit Regeltypen

Die Erweiterbarkeit der Regeln wird mit einer exemplarischen Anpassung des Quellcodes überprüft. Der neue Regeltyp `YetAnotherRule` soll hinzugefügt werden. Zunächst wurde eine neue Klasse `YetAnotherRuleAasGeneratorPipelineStep` erstellt, die von `IPipelineStep` erbt. Anschließend wurde die Klasse in der `AasGeneratorPipeline` registriert. Es waren keine weiteren Änderungen an bestehenden Klassen notwendig. Die Regressionstests wurden erfolgreich ausgeführt. Das Kriterium ist somit erfüllt.

7.3 Diskussion der Ergebnisse

Die Evaluation zeigt, dass die meisten Akzeptanzkriterien ganz oder teilweise erfüllt wurden. Bei Betrachtung der in Kapitel 4 erstellten Traceability-Matrix können die Anforderungen wie folgt bewertet werden:

FR-01 Regeltypen: Teilweise erfüllt (Filterregel nicht implementiert)

FR-02 Optionale und Pflichtfelder: Erfüllt

FR-03 Regel-Ablage: Teilweise erfüllt (Template-Qualifier noch im erzeugten Submodel)

FR-04 SME-Unterstützung: Teilweise erfüllt (**SML** nicht implementiert; **MLP** nicht implementiert)

FR-05 Typ→Instanz-Erzeugung: Teilweise erfüllt (Template-Qualifier noch im erzeugten Submodel)

FR-06 Template-Verwaltung: Teilweise erfüllt (Nicht alle **CRUD**-Operationen implementiert; Nicht Teil des AAS-Generators)

FR-07 AAS-Generator: Erfüllt

FR-08 Validierungsfehler: Erfüllt

FR-09 Logging: Erfüllt

FR-10 UI-Übersichten: Erfüllt

FR-11 UI-Diagnose: Nicht Erfüllt (Nicht Teil des AAS-Generators)

NFR-0 Konformität: Teilweise Erfüllt (Template-Qualifier noch im erzeugten Submodel)

NFR-02 Erweiterbarkeit: Erfüllt

NFR-03 Performance: Erfüllt

NFR-04 Testbarkeit: Teilweise erfüllt (Branch-Coverage nur 54.6%)

NFR-05 Sicherheit/Robustheit: Erfüllt

Damit sind fünf funktionale Anforderungen vollständig erfüllt, fünf funktionale Anforderungen teilweise erfüllt und eine funktionale Anforderung nicht erfüllt. Von den nicht-funktionalen Anforderungen sind drei vollständig erfüllt und zwei teilweise erfüllt. Der in dieser Arbeit vorgestellte Prototyp des *AAS-Generators* erfüllt somit die meisten der definierten Anforderungen. Die gewählte Pipeline-Architektur bietet mehr Struktur als der vorherige Ansatz und lässt sich sehr einfach um weitere Regeln erweitern. Die Listen-Regeln zeigen exemplarisch, wie komplexe Regeln umgesetzt werden können.

Die Filter-Regel ist aktuell noch nicht implementiert, da sie eine komplexere Logik erfordert und eine Regelsprache wie z.B. *JSONata* eingebunden werden müsste.

Die Unterstützung von *SMLs* bei den Listenregeln erfordert noch kleinere Anpassungen an die Logik, des *DuplicateCollectionsStep*, da die Elemente keine *IDs* besitzen müssen und nur durch die Reihenfolgen identifiziert werden.

Den *MLPs* wurde in dieser Arbeit keine Betrachtung gewidmet, obwohl sie bereits von der vorhandenen Lösung teilweise unterstützt wurden.

Die Template-Qualifier sind aktuell noch im erzeugten Submodel vorhanden, was nicht dem gewünschten Verhalten entspricht. Ein interessanter Ansatz wäre, die Template-Qualifiers nicht nur zu entfernen, sondern sie in geeignete Concept-Qualifier zu überführen. Diese können für die Nachvollziehbarkeit des Mappings nützliche Informationen enthalten.

Die Testabdeckung ist mit 80.0% Line-Coverage und 54.6% Branch-Coverage im Verzeichnis *MnestixCore/AasGenerator* zufriedenstellend, jedoch sind nicht alle Edge-Cases abgedeckt. Hier könnten weitere Tests hinzugefügt werden, um die Branch-Coverage zu erhöhen. In dem Zug können auch Tests für die anderen Teile des *Mnestix*-Backends hinzugefügt werden, um die Gesamt-Testabdeckung zu verbessern.

8 Fazit und Ausblick

In diesem Kapitel werden in Abschnitt 8.1 die Ergebnisse dieser Arbeit zusammengefasst, in Abschnitt 8.2 die Zielerreichung bewertet und mögliche Weiterentwicklungen 8.3, sowie in Abschnitt 8.4 alternative Ansätze vorgestellt. Abschließend wird in Abschnitt 8.5 ein Ausblick auf zukünftige Entwicklungen im Bereich der automatisierten Erstellung von AASs gegeben.

8.1 Zusammenfassung

Diese Arbeit hatte das Ziel, eine Rules-Engine zur automatisierten Erstellung von AASs im Kontext von Industrie 4.0 zu konzipieren und prototypisch zu implementieren. Das Überführen bestehender strukturierter Daten in standardisierte digitale Zwillinge stellt eine zentrale Herausforderung dar, die durch den hier verfolgten regelbasierten Ansatz adressiert wurde.

Die in Kapitel 3 durchgeführte Literatur- und Marktanalyse zeigte, dass bestehende Ansätze meist auf bestimmte Datenformate beschränkt sind oder nur eingeschränkte Möglichkeiten zur Definition komplexerer Mapping-Regeln bieten.

Auf Grundlage der in Kapitel 4 erarbeiteten Anforderungen wurde in Kapitel 5 ein umfassendes Regelwerk entwickelt, das verschiedene Regeltypen unterstützt, darunter Default-Werte, Pfadregeln, Schleifenregeln, Filter-Regeln sowie Kardinalitätsregeln.

Die prototypische Implementierung in Kapitel 6 erfolgte als Erweiterung des bestehenden *Eclipse Mnestix* AAS-Generators unter Anwendung des Pipeline-Patterns nach Buschmann et al. [54]. Diese Architektur ermöglichte eine klare Trennung der Verantwortlichkeiten und eine hohe Erweiterbarkeit für zukünftige Regeltypen. Die Evaluation in Kapitel 7 bestätigte, dass ein funktionsfähiger Prototyp vorliegt, der zentrale Regeltypen unterstützt und mit überschaubarem Aufwand um weitere erweitert werden kann.

8.2 Erreichen der Zielsetzung

Die Frage, wie AASs automatisiert und regelbasiert aus strukturierten Datenquellen generiert werden können, wurde durch die Entwicklung des Regelwerks und der Rules-Engine weitgehend beantwortet.

Die in Kapitel 7 durchgeführte Evaluation zeigt, dass der Großteil der definierten Anforderungen vollständig oder teilweise erfüllt ist.

Das Ziel, einen lauffähigen Prototypen zu entwickeln, der die Kernfunktionalität mit Pfad- und Schleifenregeln demonstriert, wurde erreicht.

8.3 Mögliche Weiterentwicklungen

Die entwickelte Rules-Engine bietet eine solide Grundlage für verschiedene Erweiterungen und Verbesserungen:

8.3.1 Vollständige AAS-Generierung

Aktuell erstellt die Rules-Engine nur Submodels. Eine naheliegende Erweiterung wäre die Generierung vollständiger **AASs** inklusive Asset-Informationen und mehrerer Submodels. Dies würde eine übergeordnete Template-Struktur erfordern, die die Beziehungen zwischen verschiedenen Submodels definiert.

8.3.2 Erweiterte Regeltypen

Basierend auf den identifizierten Limitationen sollten zunächst die fehlenden Filter-Regeln implementiert werden, was die Integration einer Regelsprache wie *JSONata* erfordert. Zusätzlich muss die Unterstützung für **SMLs** und **MLPs** vervollständigt werden. Darüber hinaus könnten Regeltypen wie Aggregationsregeln, semantische Validierungsregeln und Verknüpfungsregeln hinzugefügt werden, um die Flexibilität und Ausdruckskraft des Regelwerks zu erhöhen.

8.3.3 Bearbeitung existierender Submodels

Der aktuelle Prototyp generiert neue Submodels basierend auf Templates. Eine sinnvolle Erweiterung wäre die Möglichkeit, bestehende Submodels zu bearbeiten und zu aktualisieren. Dies könnte durch die Einführung von Update-Regeln erfolgen, die es ermöglichen, nur bestimmte Felder zu ändern oder hinzuzufügen, ohne das gesamte Submodel neu generieren zu müssen.

8.3.4 Nachvollziehbarkeit

Nach der Erstellung eines Submodels wäre es hilfreich, eine Historie der angewendeten Regeln und Änderungen zu speichern. Dies könnte in Concept-Qualifiern erfolgen, um die Nachvollziehbarkeit der generierten **AASs** zu verbessern.

8.3.5 Verbesserte Benutzeroberfläche

Die aktuelle **GUI** im *Eclipse Mnestix*-Browser bietet grundlegende Funktionalitäten. Eine erweiterte Benutzeroberfläche könnte folgende Features umfassen:

- Visual Rule Designer bzw. Low-Code-Plattform
- Debugging-Tools zur Nachverfolgung der Regelausführung
- Template-Versionierung und Vergleichsfunktionen

8.3.6 Erweiterte Datenverarbeitung

Um die aktuelle Beschränkung auf **JSON**-Daten zu überwinden und die Konformität zu verbessern, könnte die Architektur erweitert werden:

- Native Unterstützung verschiedener Eingabeformate (**XML**, **Comma-Separated Values (CSV)**)
- Native Unterstützung von Industrie 4.0 relevanten Datenquellen (z.B. **OPC UA**, **MQTT**)
- Unterstützung des **AASX**-Formats für den direkten Import von **AASs**-Templates

8.4 Alternative Ansätze

Neben dem in dieser Arbeit verfolgten deterministischen, regelbasierten Ansatz existieren weitere vielversprechende Technologien, die in zukünftigen Arbeiten untersucht werden könnten:

8.4.1 Semantisches Mapping

Der von *Stohmeier et al.* [22] vorgestellte Ansatz mit **SIPs** bietet interessante Möglichkeiten für die semantische Integration verschiedener Datenquellen. Durch die Verwendung von Semantic IDs aus standardisierten Katalogen wie der **IEC 61360**-Datenbank [24] könnte eine automatische Zuordnung von Datenfeldern zu **AAS**-Elementen erfolgen, ohne dass manuell Regeln definiert werden müssen.

8.4.2 KI-gestützte Ansätze

Die aktuellen Entwicklungen im Bereich der **KI**, insbesondere bei Large Language Models (LLMs), eröffnen neue Möglichkeiten für die automatisierte **AAS**-Generierung. Der von *Zhao et al.* [21] vorgestellte semi-automatische Ansatz zeigt bereits das Potenzial von **KI**-Methoden für die Extraktion von Informationen aus unstrukturierten Datenquellen mittels **NLP**.

In Verbindung mit der in dieser Arbeit entwickelten regelbasierten Methode könnte ein hybrider Ansatz verfolgt werden, bei dem **KI**-Modelle genutzt werden, um initiale Regelvorschläge zu generieren, die dann manuell verfeinert und deterministisch ausgeführt werden.

8.5 Ausblick

Um auch in Zukunft international wettbewerbsfähig zu bleiben, muss die deutsche Industrie einen tiefgreifenden Wandel hin zu vernetzten und hoch automatisierten Produktionsabläufen durchlaufen. Dieser Wandel eröffnet erhebliche Chancen, stellt Unternehmen jedoch zugleich vor große Herausforderungen. Unabhängig davon, ob sich die **AAS** oder ein alternativer Standard als dominierende Technologie für digitale Zwillinge durchsetzt, bleibt die Transformation bestehender Daten in standardisierte digitale Repräsentationen eine Schlüsselaufgabe. Je nach Anwendungsfall werden dabei unterschiedliche Lösungsansätze ihre Stärken ausspielen.

Der in dieser Arbeit vorgestellte regelbasierte Ansatz leistet zusammen mit anderen Open-Source-Projekten einen Beitrag, um die von Organisationen wie der **IDTA** verfolgte Etablierung offener, interoperabler Lösungen voranzubringen [12]. Nur durch breit verfügbare und standardkonforme Werkzeuge lässt sich das volle Potenzial von Industrie 4.0 ausschöpfen.

Anhang

In diesem [Github-Repository](#) sind alle relevanten Ressourcen, die im Rahmen dieser Arbeit erstellt wurden, abgelegt. Dazu gehören:

- Der Quellcode der entwickelten Template Engine, inklusive aller Implementierungen und Unit Tests.
- Weitere Diagramme zu der Architektur und dem Datenfluss.
- Alle in der Arbeit in Kurzform dargestellten Submodel-Templates als [JSON](#)-Dateien.
- Die *Insomnia*-Sammlung mit Anfragen zur Evaluierung der Rules-Engine.

Erklärung zur Abschlussarbeit

Hiermit versichere ich, die eingereichte Abschlussarbeit selbständig verfasst und keine andere als die von mir angegebenen Quellen und Hilfsmittel benutzt zu haben. Wörtlich oder inhaltlich verwendete Quellen wurden entsprechend den anerkannten Regeln wissenschaftlichen Arbeitens zitiert. Für die Vervollständigung von einzelnen Sätzen habe ich die KI-Tools ChatGPT und Claude Sonnet 4 verwendet. Ich erkläre weiterhin, dass die vorliegende Arbeit noch nicht anderweitig als Abschlussarbeit eingereicht wurde. Das Merkblatt zum Täuschungsverbot im Prüfungsverfahren der Technischen Hochschule Augsburg habe ich gelesen und zur Kenntnis genommen. Ich versichere, dass die von mir abgegebene Arbeit keinerlei Plagiate, Texte oder Bilder umfasst, die durch von mir beauftragte Dritte erstellt wurden.

Augsburg, den 21. September 2025



Luis Schweinberger

Abkürzungsverzeichnis

AAS Asset Administration Shell. 2–5, 7–10, 12–20, 22–25, 27, 28, 35, 43, 51, 54, 55, 58–61
AASX Asset Administration Shell Datei-Format. 7, 15, 20, 23, 60
AC Acceptance Criterion. 18, 26, 54
AML Automation Modelling Language. 14
API Application Programming Interface. 7, 10, 19, 20, 22–26, 37, 39, 43, 51, 54

BMBF Bundesministerium für Bildung und Forschung. 2
BMFTR Bundesministerium für Forschung, Technologie und Raumfahrt. 2
BMWE Bundesministerium für Wirtschaft und Energie. 2
BMWK Bundesministerium für Wirtschaft und Klimaschutz. 2

CRUD Create, Read, Update, Delete. 22, 56
CSV Comma-Separated Values. 60

DTDL Digital Twin Definition Language. 14

ETL Extract, Transform, Load. 19
EU DPP EU Digital Product Passport. 2, 8

GUI Graphical User Interface. 13, 19, 23, 25, 51, 60

HTTP Hypertext Transfer Protocol. 16

ID Identifier. 10, 13, 15, 19, 20, 22, 24, 25, 36, 43, 54, 57
IDTA Industrial Digital Twin Association. 2, 7, 8, 10, 51, 55, 61
IEC International Electrotechnical Commission. 13, 14, 60
IESE Institut für Experimentelles Software Engineering. 9, 15
IOSB Institut für Optronik, Systemtechnik und Bildauswertung. 16
IT Information Technology. 16

JSON JavaScript Object Notation. 10, 13, 19, 20, 22, 25, 28, 54, 60, 62

KI Künstliche Intelligenz. 12, 13, 61

MLP MultiLanguageProperty. 8, 22, 56, 57, 59
MQTT Message Queuing Telemetry Transport. 15, 60

NLP Natural Language Processing. 12, 61

OPC UA Open Platform Communications Unified Architecture. 13–16, 60

OT Operational Technology. 16

PDF Portable Document Format. 12, 20

PLC Programmable Logic Controller. 14

RAMI4.0 Reference Architecture Model Industrie 4.0. 4, 5, 7

RDF Resource Description Framework. 13

REST Representational State Transfer. 10, 15, 16, 19, 20, 22, 23, 51

SDK Software Development Kit. 13, 15

SIP Semantic Integration Pattern. 13, 60

SMC SubmodelElementCollection. 8, 18, 22, 28, 31, 32, 51, 52, 55, 66

SME SubmodelElement. 8–10, 15, 18, 22, 24, 25, 27–31, 33, 49, 51

SML SubmodelElementList. 8, 18, 22, 56, 57, 59

UC Use Case. 18, 20, 21, 26

XML Extensible Markup Language. 7, 12–14, 20, 23, 60

XSD XML Schema Definition. 13

Abbildungsverzeichnis

2.1	Objektwelten des RAMI4.0 [11]	5
2.2	Drei-Achsen-Modell des RAMI4.0 [11]	6
2.3	Aufbau des Mnestix-Projekts [17]	9
2.4	Dataflow in Mnestix [19]	11
4.1	Use-Case-Übersicht der Rules-Engine	21
6.1	Klassendiagramm der Pipeline-Komponenten	42
6.2	Klassendiagramm der AAS-Generator-Komponenten	44
6.3	Ablauf der Submodel-Generierung im Eclipse-Mnestix-Backend	45
6.4	Ansicht einer Property eines Submodel-Templates im AAS Browser [50]	51
6.5	Neue Ansicht einer SMC eines Submodel-Templates im AAS Browser [50]	52

Tabellenverzeichnis

4.1	Stakeholderübersicht	19
4.2	Traceability-Matrix	26

Codeverzeichnis

5.1	Default-Werte – Template-Submodel	29
5.2	Default-Werte – Instanz-Submodel	29
5.3	Pfadregel – Payload	29
5.4	Pfadregel – Template-Submodel	30
5.5	Pfadregel – Instanz-Submodel	30
5.6	Listenregel – Payload	30
5.7	Listenregel – Template-Submodel	31
5.8	Listenregel – Instanz-Submodel	31
5.9	Filterregel – Payload	32
5.10	Filterregel – Template-Submodel	32
5.11	Filterregel – Instanz-Submodel	32
5.12	Cardinality – Payload	33
5.13	Kardinalitätsregel – Template-Submodel	33
5.14	Kardinalitätsregel – Instanz-Submodel	34
6.1	Pipeline-Kontext	37
6.2	Pipeline	38
6.3	IPipelineStep-Interface	38
6.4	Strukturierte Fehlerbehandlung	39
6.5	Pipeline-Builder	40
6.6	SubModelDataToInstanceMapper	41
6.7	Dateistruktur des AAS Generator	43
6.8	Submodel-Template	46
6.9	JSON-Eingabe-Payload	47
6.10	Gewünschtes Submodel	48
6.11	Gewünschtes Submodel	50

Literaturverzeichnis

- [1] H. Kagermann, W.-D. Lukas und W. Wahlster, „Industrie 4.0: Mit Dem Internet Der Dinge Auf Dem Weg Zur 4. Industriellen Revolution“, *VDI-Nachrichten*, Nr. 13-2011, S. 2, besucht am 5. Mai 2025. Adresse: https://www.dfki.de/fileadmin/user_upload/DFKI/Medien/News_Media/Presse/Presse-Highlights/vdinach2011a13-ind4.0-Internet-Dinge.pdf (siehe S. 2).
- [2] *Hintergrund zur Plattform Industrie 4.0*. besucht am 4. Juni 2025. Adresse: <https://www.plattform-i40.de/IP/Redaktion/DE/Standardartikel/hintergrund-plattform.html> (siehe S. 2).
- [3] *Was ist Industrie 4.0?* Besucht am 17. Sep. 2025. Adresse: <https://www.plattform-i40.de/IP/Navigation/DE/Industrie40/WasIndustrie40/was-ist-industrie-40.html> (siehe S. 2).
- [4] W. Kritzinger, M. Karner, G. Traar, J. Henjes und W. Sihn, „Digital Twin in manufacturing: A categorical literature review and classification“, *IFAC-PapersOnLine*, Jg. 51, Nr. 11, S. 1016–1022, 2018. doi: [10.1016/j.ifacol.2018.08.474](https://doi.org/10.1016/j.ifacol.2018.08.474). besucht am 1. Sep. 2025. Adresse: <https://linkinghub.elsevier.com/retrieve/pii/S2405896318316021> (siehe S. 2, 4, 12).
- [5] *Leitbild 2030 Industrie 4.0*, Mai 2019 (siehe S. 2).
- [6] A. Preut, J.-P. Kopka und U. Clausen, „Digital Twins for the Circular Economy“, *Sustainability*, Jg. 13, Nr. 18, S. 10467, Sep. 2021. doi: [10.3390/su131810467](https://doi.org/10.3390/su131810467). besucht am 17. Sep. 2025. Adresse: <https://www.mdpi.com/2071-1050/13/18/10467> (siehe S. 2).
- [7] U. European, *EU's Digital Product Passport: Advancing Transparency and Sustainability* | *Data.Europa.Eu*, Sep. 2024. besucht am 17. Sep. 2025. Adresse: <https://data.europa.eu/en/news-events/news/eus-digital-product-passport-advancing-transparency-and-sustainability> (siehe S. 2).
- [8] *Regulation - EU - 2024/1781 - EN - EUR-Lex*, Juni 2024. besucht am 17. Sep. 2025. Adresse: <https://eur-lex.europa.eu/eli/reg/2024/1781/oj/eng> (siehe S. 2).
- [9] *Asset Administration Shell Specification - Part 1: Metamodel*, März 2025. besucht am 23. Apr. 2025. Adresse: https://industrialdigitaltwin.org/wp-content/uploads/2025/03/IDTA-01001-3-0-2_SpecificationAssetAdministrationShell_Part1_Metamodel.pdf (siehe S. 2, 7, 8, 18, 20, 23, 28, 55).
- [10] E. Negri, L. Fumagalli und M. Macchi, „A Review of the Roles of Digital Twin in CPS-based Production Systems“, *Procedia Manufacturing*, Jg. 11, S. 939–948, 2017. doi: [10.1016/j.promfg.2017.07.198](https://doi.org/10.1016/j.promfg.2017.07.198). besucht am 5. Sep. 2025. Adresse: <https://linkinghub.elsevier.com/retrieve/pii/S2351978917304067> (siehe S. 4).
- [11] *Referenzarchitekturmodell Industrie 4.0 (RAMI4.0)*, Apr. 2016. besucht am 13. Mai 2025. Adresse: <https://www.beuth.de/en/technical-rule/din-spec-91345/250940128> (siehe S. 5–7, 66).
- [12] *IDTA – Der Standard Für Den Digitalen Zwilling - Startseite*. besucht am 5. Sep. 2025. Adresse: <https://industrialdigitaltwin.org/> (siehe S. 7, 61).
- [13] *AAS Spezifikationen Archiv*. besucht am 5. Sep. 2025. Adresse: <https://industrialdigitaltwin.org/content-hub/aasspecifications> (siehe S. 7).

- [14] J. Zhang, C. Ellwein, M. Heithoff, J. Michael und A. Wortmann, „Digital twin and the asset administration shell: An Analysis of the Three Types of AASs and their Feasibility for Digital Twin Engineering“, *Software and Systems Modeling*, Jg. 24, Nr. 3, S. 771–793, Juni 2025. doi: [10.1007/s10270-024-01255-0](https://doi.org/10.1007/s10270-024-01255-0). besucht am 5. Sep. 2025. Adresse: <https://link.springer.com/10.1007/s10270-024-01255-0> (siehe S. 7).
- [15] M. Jacoby, M. Baumann, T. Bischoff, H. Mees, J. Müller, L. Stojanovic und F. Volz, „Open-Source Implementations of the Reactive Asset Administration Shell: A Survey“, *Sensors*, Jg. 23, Nr. 11, S. 5229, Mai 2023. doi: [10.3390/s23115229](https://doi.org/10.3390/s23115229). besucht am 24. Apr. 2025. Adresse: <https://www.mdpi.com/1424-8220/23/11/5229> (siehe S. 7, 14–16).
- [16] *Teilmodelle - Industrial Digital Twin Association e. V.* besucht am 7. Juli 2025. Adresse: <https://industrialdigitaltwin.org/content-hub/teilmodelle> (siehe S. 8).
- [17] *Eclipse Mnestix – Open-Source-Software - XITASO.* besucht am 7. Juli 2025. Adresse: <https://xitaso.com/kompetenzen/mnestix/> (siehe S. 9, 10, 66).
- [18] A. Zielstorff, *BaSyx Explained - Eclipse BaSyx.* besucht am 2. Sep. 2025. Adresse: https://wiki.basysx.org/en/latest/content/introduction/basysx_explained.html (siehe S. 10, 15).
- [19] A. Hoffmann, M. Hofer und X. G. I. bibinitperiod S. Solutions, *Eclipse Mnestix - The Opensource AAS-Ecosystem*, Aug. 2025. besucht am 6. Sep. 2025 (siehe S. 10, 11, 66).
- [20] B. J. Oakes, A. Parsai, B. Meyers, I. David, S. V. Mierlo, S. Demeyer, J. Denil, P. D. Meulenaere und H. Vangheluwe, „A Digital Twin Description Framework and Its Mapping to Asset Administration Shell“, in *Model-Driven Engineering and Software Development*, L. F. Pires, S. Hammoudi und E. Seidewitz, Hrsg., Bd. 1708, Cham: Springer Nature Switzerland, 2023, S. 1–24. doi: [10.1007/978-3-031-38821-7_1](https://doi.org/10.1007/978-3-031-38821-7_1). besucht am 24. Apr. 2025. Adresse: https://link.springer.com/10.1007/978-3-031-38821-7_1 (siehe S. 12).
- [21] J. Zhao, B. Vogel-Heuser, F. Bi, J. Höfgen, F. Ocker, B. Vojanec, T. Markert und A. Kraft, „A Semi-Automatic Approach for Asset Administration Shell Creation from Heterogeneous Data“, *IFAC-PapersOnLine*, Jg. 56, Nr. 2, S. 3673–3679, 2023. doi: [10.1016/j.ifacol.2023.10.1532](https://doi.org/10.1016/j.ifacol.2023.10.1532). besucht am 24. Apr. 2025. Adresse: <https://linkinghub.elsevier.com/retrieve/pii/S2405896323019407> (siehe S. 12, 61).
- [22] F. Strohmeier, G. Güntner, D. Glachs und R. Mayr, „Semantic Integration Patterns for Industry 4.0:“ In *Proceedings of the 3rd International Conference on Innovative Intelligent Industrial Production and Logistics*, Valletta, Malta: SCITEPRESS - Science and Technology Publications, 2022, S. 197–205. doi: [10.5220/0011550100003329](https://doi.org/10.5220/0011550100003329). besucht am 18. Aug. 2025. Adresse: <https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0011550100003329> (siehe S. 13, 60).
- [23] G. Güntner, D. Glachs, S. Linecker und F. Strohmeier, „Industrial Communication with Semantic Integration Patterns“, in *Innovative Intelligent Industrial Production and Logistics*, S. Terzi, K. Madani, O. Gusikhin und H. Panetto, Hrsg., Bd. 1886, Cham: Springer Nature Switzerland, 2023, S. 319–335. doi: [10.1007/978-3-031-49339-3_20](https://doi.org/10.1007/978-3-031-49339-3_20). besucht am 2. Mai 2025. Adresse: https://link.springer.com/10.1007/978-3-031-49339-3_20 (siehe S. 13).
- [24] *IEC 61360 Elektrische/Elektronische Bauteile - IEC-Datenbanken - VDE VERLAG.* besucht am 1. Sep. 2025. Adresse: <https://www.vde-verlag.de/iec-normen/iec-datenbanken/iec-61360-elektronische-bauteile.html> (siehe S. 13, 60).
- [25] J. Arm, T. Benesl, P. Marcon, Z. Bradac, T. Schröder, A. Belyaev, T. Werner, V. Braun, P. Kamensky, F. Zezulka, C. Diedrich und P. Dohnal, „Automated Design and Integration of Asset Administration Shells in Components of Industry 4.0“, *Sensors*, Jg. 21, Nr. 6, S. 2004, März 2021. doi: [10.3390/s21062004](https://doi.org/10.3390/s21062004). besucht am 24. Apr. 2025. Adresse: <https://www.mdpi.com/1424-8220/21/6/2004> (siehe S. 13).
- [26] *Unified Architecture - Landingpage.* besucht am 1. Sep. 2025. Adresse: <https://opcfoundation.org/about/opc-technologies/opc-ua/> (siehe S. 13).

- [27] N. Braunisch, M. Ristin-Kaufmann, R. Lehmann, M. Wollschlaeger und H. W. Van De Venn, „Generation of Digital Twins for Information Exchange Between Partners in the Industrie 4.0 Value Chain“, in *2023 IEEE 21st International Conference on Industrial Informatics (INDIN)*, Lemgo, Germany: IEEE, Juli 2023, S. 1–6. DOI: [10.1109/INDIN51400.2023.10218306](https://doi.org/10.1109/INDIN51400.2023.10218306). besucht am 24. Apr. 2025. Adresse: <https://ieeexplore.ieee.org/document/10218306/> (siehe S. 13).
- [28] S. Cavalieri und M. G. Salafia, „Asset Administration Shell for PLC Representation Based on IEC 61131–3“, *IEEE Access*, Jg. 8, S. 142 606–142 621, 2020. DOI: [10.1109/ACCESS.2020.3013890](https://doi.org/10.1109/ACCESS.2020.3013890). besucht am 23. Apr. 2025. Adresse: <https://ieeexplore.ieee.org/document/9154696/> (siehe S. 14).
- [29] C. Schmidt, F. Volz, L. Stojanovic und G. Sutschet, „Increasing Interoperability between Digital Twin Standards and Specifications: Transformation of DTDL to AAS“, *Sensors*, Jg. 23, Nr. 18, S. 7742, Sep. 2023. DOI: [10.3390/s23187742](https://doi.org/10.3390/s23187742). besucht am 24. Apr. 2025. Adresse: <https://www.mdpi.com/1424-8220/23/18/7742> (siehe S. 14).
- [30] baanders, *DTDL models - Azure Digital Twins*. besucht am 25. Aug. 2025. Adresse: <https://learn.microsoft.com/en-us/azure/digital-twins/concepts-models> (siehe S. 14).
- [31] A. Zielstorff, D. Schöttke, A. Hohenhövel, T. Kämpfe, S. Schäfer und F. Schnicke, „Harmonizing Heterogeneity: A Novel Architecture for Legacy System Integration with Digital Twins in Industry 4.0“, in *Innovative Intelligent Industrial Production and Logistics*, S. Terzi, K. Madani, O. Gusikhin und H. Panetto, Hrsg., Bd. 1886, Cham: Springer Nature Switzerland, 2023, S. 68–87. DOI: [10.1007/978-3-031-49339-3_5](https://doi.org/10.1007/978-3-031-49339-3_5). besucht am 19. Aug. 2025. Adresse: https://link.springer.com/10.1007/978-3-031-49339-3_5 (siehe S. 14).
- [32] A. Luder, A.-K. Behnert, F. Rinker und S. Biffl, „Generating Industry 4.0 Asset Administration Shells with Data from Engineering Data Logistics“, in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vienna, Austria: IEEE, Sep. 2020, S. 867–874. DOI: [10.1109/ETFA46521.2020.9212149](https://doi.org/10.1109/ETFA46521.2020.9212149). besucht am 24. Apr. 2025. Adresse: <https://ieeexplore.ieee.org/document/9212149/> (siehe S. 14).
- [33] *What is AutomationML? – AutomationML*. besucht am 1. Sep. 2025. Adresse: <https://www.automationml.org/about-automationml/automationml/> (siehe S. 14).
- [34] A. Orzelski und M. Hoffmeister, *Eclipse-Aaspe/Server*, Eclipse AASX Package Explorer and Server, Juli 2025. besucht am 2. Sep. 2025. Adresse: <https://github.com/eclipse-aaspe/server> (siehe S. 15).
- [35] M. T. Delgado, *Eclipse AASX Package Explorer and Server* | *projects.eclipse.org*, März 2022. besucht am 2. Sep. 2025. Adresse: <https://projects.eclipse.org/projects/dt.aaspe> (siehe S. 15).
- [36] A. Zielstorff und R. Ficher, *Eclipse BaSyx*. besucht am 4. Sep. 2025. Adresse: <https://github.com/eclipse-basyx> (siehe S. 15).
- [37] A. Zielstorff, *DataBridge Component - Eclipse BaSyx*. besucht am 2. Sep. 2025. Adresse: https://wiki.basyx.org/en/latest/content/user_documentation/basyx_components/databridge/index.html (siehe S. 15).
- [38] F. Schnicke, *Device-Integration mit Eclipse BaSyx einfach machen*, März 2023. besucht am 2. Sep. 2025. Adresse: <https://www.iese.fraunhofer.de/blog/device-integration-mit-basyx/> (siehe S. 15).
- [39] R. Yokota, *JSONata Documentation · JSONata*, Aug. 2025. besucht am 7. Sep. 2025. Adresse: <http://docs.jsonata.org/> (siehe S. 15).
- [40] M. Ghazanfar Ali Danish, *Eclipse-Basyx/Basyx-Databridge*, Eclipse BaSyx, Juli 2025. besucht am 2. Sep. 2025. Adresse: <https://github.com/eclipse-basyx/basyx-databridge> (siehe S. 16).
- [41] L. Stojanovic und M. Jacoby, *FA³ST Ecosystem: KI-gestützte Entwicklung und Nutzung von Verwaltungsschalen - Fraunhofer IOSB*. besucht am 2. Sep. 2025. Adresse: <https://www.iosb.fraunhofer.de/de/projekte-produkte/faaast-ecosystem-digitale-zwillinge-aas.html> (siehe S. 16).

- [42] M. Jacoby, *FraunhoferIOSB/FAAAS-Service: FA³ST - Fraunhofer Advanced Asset Administration Shell Tools (for Digital Twins)*. besucht am 2. Sep. 2025. Adresse: <https://github.com/FraunhoferIOSB/FAAAS-Service> (siehe S. 16).
- [43] M. Reis, A. Lourenço, M. A. Maamari, P. Maló, G. Di Orio und F. Marques, *NOVAAS Collection / NOVAAS Catalog / NOVA School of Science and Technology / NOVA Asset Administration Shell · GitLab*, Nova School of Science and Technology, Juli 2025. besucht am 3. Sep. 2025. Adresse: <https://gitlab.com/novaas/catalog/nova-school-of-science-and-technology/novaas> (siehe S. 16).
- [44] G. Di Orio, P. Malo und J. Barata, „NOVAAS: A Reference Implementation of Industrie4.0 Asset Administration Shell with best-of-breed practices from IT engineering“, in *IECON 2019 - 45th Annual Conference of the IEEE Industrial Electronics Society*, Lisbon, Portugal: IEEE, Okt. 2019, S. 5505–5512. doi: [10.1109/IECON.2019.8927081](https://doi.org/10.1109/IECON.2019.8927081). besucht am 3. Sep. 2025. Adresse: <https://ieeexplore.ieee.org/document/8927081/> (siehe S. 16).
- [45] B. Hardill, S. McLaughlin, G. Dandele, K. Yokoi, H. Nishiyama und M. Bonani, *Low-Code Programming for Event-Driven Applications : Node-RED*. besucht am 3. Sep. 2025. Adresse: <https://nodered.org/> (siehe S. 16).
- [46] G. Di Orio, M. Reis und F. Marques, *NOVAAS Collection / NOVAAS Catalog / NOVA School of Science and Technology / NOVAAS Hydraulic Unit Simulation · GitLab*, Juli 2025. besucht am 4. Sep. 2025. Adresse: <https://gitlab.com/novaas/catalog/nova-school-of-science-and-technology/novaas-hydraulic-unit-simulation> (siehe S. 16).
- [47] *Systems and Software Engineering Life Cycle Processes — Requirements Engineering*, S.67-74, Nov. 2018. besucht am 5. Sep. 2025. Adresse: <https://drkasbokar.com/wp-content/uploads/2024/09/29148-2018-ISOIECIEEE.pdf> (siehe S. 18).
- [48] K. McCaffrey, *ISO/IEC/IEEE 29148 Systems and Software Requirements Specification (SRS) Example Template*. besucht am 5. Sep. 2025. Adresse: <https://www.well-architected-guide.com/documents/iso-iec-ieee-29148-template/> (siehe S. 18).
- [49] C. Ocando Röhrich, *Mnestix Personas*, Jan. 2025. besucht am 8. Juli 2025 (siehe S. 19).
- [50] Xitaso, *Eclipse-Mnestix/Mnestix-Browser*, Eclipse Mnestix, Juli 2025. besucht am 7. Juli 2025. Adresse: <https://github.com/eclipse-mnestix/mnestix-browser> (siehe S. 20, 23, 51, 52).
- [51] *Systems and software engineering. Systems and software quality requirements and evaluation (SQaRE). System and software quality models*: März 2011. doi: [10.3403/30215101](https://doi.org/10.3403/30215101). besucht am 17. Sep. 2025. Adresse: <https://linkresolver.bsigroup.com/junction/resolve/00000000030215101?restype=standard> (siehe S. 21).
- [52] J. Hirschmüller, *Mnestix Quality Refinement*, Sep. 2025. besucht am 18. Sep. 2025 (siehe S. 21).
- [53] A. Zahler, *Jsonata-Js/Jsonata: JSONata Query and Transformation Language - http://jsonata.org*. besucht am 7. Sep. 2025. Adresse: <https://github.com/jsonata-js/jsonata> (siehe S. 28).
- [54] „Pipes-And-Filters“, in *Pattern-orientierte Softwarearchitektur: ein Pattern-System*, F. Buschmann, Hrsg., Nachdr., München: Addison-Wesley, 2002, S. 54–71 (siehe S. 35, 36, 58).
- [55] H. Karabulut, *Pipeline Pattern in .NET Core: Building Modular and Flexible Workflows* | Medium, Nov. 2024. besucht am 7. Sep. 2025. Adresse: <https://karabuluthakan.medium.com/pipeline-pattern-in-net-core-building-modular-and-flexible-workflows-7ec36f33ce90> (siehe S. 35, 36).
- [56] S. Toub, *ConfigureAwait FAQ*, Dez. 2019. besucht am 11. Sep. 2025. Adresse: <https://devblogs.microsoft.com/dotnet/configureawait-faq/> (siehe S. 38).
- [57] G. J. Myers, *The Art of Software Testing*, 2nd ed. Hoboken, N.J: J. Wiley & Sons, 2004 (siehe S. 55).

- [58] S. Heppner, *Aas-Specs-Metamodel/Schemas/Json at Master · Admin-Shell-Io/Aas-Specs-Metamodel*. besucht am 16. Sep. 2025. Adresse: <https://github.com/admin-shell-io/aas-specs-metamodel/tree/master/schemas/json> (siehe S. 55).